МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫХ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»
Факультет информационных технологий
Кафедра информационных систем и технологий
Специальность 6-05-0611-01 «Информационные системы и технологии»
Дисциплина Операционные системы
Лабораторная работа № 2

РАБОТА В КОМАНДНОЙ ОБОЛОЧКЕ

Часть 1. Linux (Arch Linux ARM)

Студент 3 курса 2 группы
Марушко Т. Ф.
Проверил: _____

Минск 2026

Минск, 2026

ОГЛАВЛЕНИЕ

Оглавление формируется средствами Word: вкладка «Ссылки» - «Оглавление» - «Автособираемое оглавление». Для этого заголовки глав размечены стилем Глава_диплома, разделов - стилем Раздел_диплома.

ВВЕДЕНИЕ

Операционная система предоставляет пользователю интерфейс взаимодействия с аппаратными ресурсами компьютера. Наиболее эффективным интерфейсом работы в системе является командная оболочка, обеспечивающая выполнение операций над файлами и каталогами, запуск программ и автоматизацию повторяющихся действий.

В настоящей лабораторной работе выполняются задания уровней 4-9 методических указаний для операционной системы Linux. Работа выполнена в дистрибутиве Arch Linux ARM в командной оболочке bash. Все задания Windows-части сопоставлены с соответствующими средствами Linux, что позволяет представить единый отчёт по лабораторной работе.

Цель работы - освоить приёмы работы с командной оболочкой и научиться составлять собственные сценарии автоматизации.

Для достижения цели поставлены следующие задачи:

- выполнить задания уровней 4-5 по навигации, копированию, просмотру и удалению файлов;
- выполнить задания уровней 6-7 по расширенному копированию, работе с консольным вводом и выводом, составлению сценариев с параметрами;
- исследовать переменные окружения и определить наиболее значимые из них для работы в консоли;
- выполнить задания уровней 8-9 по подсчёту команд в PATH, выводу дерева каталогов с размерами и мониторингу процессов.

Выполнение всех заданий сопровождалось фиксацией результата: для каждой команды сделан снимок полноэкранной консоли и сохранён текстовый вывод. Полнота выполнения проверена автоматизированным распознаванием текста на снимках.

СРЕДА ВЫПОЛНЕНИЯ

Работа выполнена на виртуальной машине с операционной системой Arch Linux ARM. Ниже приведены основные характеристики системы, полученные командами uname и чтением файла /etc/os-release.

Параметры системы

Для получения сведений о системе выполняется команда uname с ключами -a, -r и -m, а также выводятся данные из файла /etc/os-release.

```
        Linux alarm 7.2.8-1-aarch64-ARCH #1 SMP PREEMPT_DYNAMIC Sat Sep 26 12:05:48 UTC 2026
  aarch64 GNU/Linux
        7.2.8-1-aarch64-ARCH
        aarch64
        NAME=«Arch Linux ARM»
        PRETTY_NAME=«Arch Linux ARM»
        ID=archarm
```

Из вывода следует, что система построена на базе Arch Linux ARM, ядро версии 7.2.8-1-aarch64-ARCH работает на архитектуре aarch64 (64-разрядная архитектура ARM). Имя узла - alarm.

СООТВЕТСТВИЕ КОМАНД WINDOWS И LINUX

Каждому заданию Windows-части методических указаний сопоставлено средство Linux, приведённое в таблице 3.1. Изучение отличий в наименовании и поведении команд позволяет освоить перенос типовых операций администрирования на unix-системы.

Таблица 3.1 - Соответствие команд Windows и Linux

Команда Windows	Средство Linux	Отличия и примечания
cd	cd	Совпадает; переход по абсолютному или относительному пути.
md	mkdir	mkdir -p создаёт и промежуточные каталоги.
rd	rmdir, rm -r	rmdir удаляет только пустой каталог, rm -r - любой.
dir	ls -R	Рекурсивный вывод вложенных элементов.
copy	cp	cp -i запрашивает подтверждение перезаписи.
copy con	cat > файл	Ввод с клавиатуры до сигнала конца файла.
move	mv	Совпадает.
ren	mv	Переименование выполняется перемещением.
fc	diff	Вывод различий двух файлов.
del	rm	rm -i запрашивает подтверждение удаления.
xcopy	cp -r	cp -rp сохраняет атрибуты файлов.
replace	cp -f	Замена файлов копированием с ключом -f.
attrib	chmod	Права доступа задаются в восьмеричной записи.
type	cat	Совпадает.
find	grep	Поиск по содержимому файлов.
echo	echo	Вывод строки, в том числе с перенаправлением.
sort	sort	Совпадает.
systeminfo	uname, cat	Данные о системе читаются из /proc и /sys.
set	env, printenv	env без параметров показывает всё окружение.
bat_01	bash bat_01.sh	Данные берутся из переменных окружения.
bat_02	bash bat_02.sh	Расширения параметров заменены на basename и stat.
bat_03	bash bat_03.sh	Параметры передаются как позиционные: $1, $2, $@.
bat_04	bash bat_04.sh	Арифметика выполняется командой bc или $(( )).
bat_05	bash bat_05.sh	Режимы работы задаются первым аргументом.

Наиболее существенные отличия связаны с отсутствием в unix-системах расширенных атрибутов файлов в понимании Windows, с различиями в разделителе элементов пути, в механизмах перенаправления вывода и в системе прав доступа, построенной на владельце и группе.

4 ВЫПОЛНЕНИЕ ЗАДАНИЙ УРОВНЕЙ 4-5

Задания уровней 4-5 выполнены в консоли Arch Linux ARM. Каждое задание сопоставлено с соответствующей командой Windows из методических указаний, приведены отличия в наименовании и поведении. Все команды вводились в полноэкранной консоли, результат каждой зафиксирован снимком экрана и сохранён текстовым выводом.

4.1 Навигация и создание каталогов

4.1.1 Переход в каталог task_01i/Русский по абсолютному пути

В соответствии с заданием Л4-5 п.1.1 выполняется команда:

```
cd ~/OS2/task_01i/Русский && pwd
```

Соответствующая команда Windows: cd

Вывод команды:

```
/home/User-6d4ec851/OS2/task_01i/Русский
```

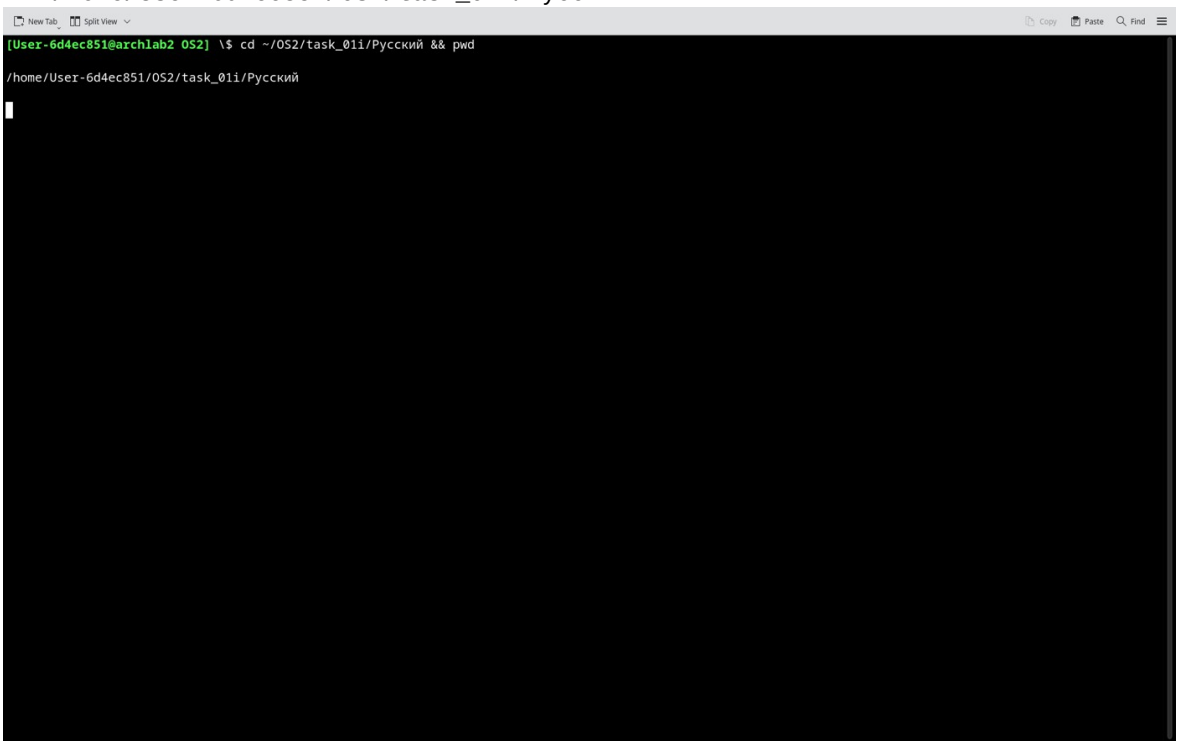


Рисунок 1 - Л4-5 п.1.1, аналог команды cd

4.1.2 Переход в каталог task_01i/English по относительному пути

В соответствии с заданием Л4-5 п.1.2 выполняется команда:

```
cd ~/OS2/task_01i/Русский && cd ../English && pwd
```

Соответствующая команда Windows: cd

Вывод команды:

```
/home/User-6d4ec851/OS2/task_01i/English
```

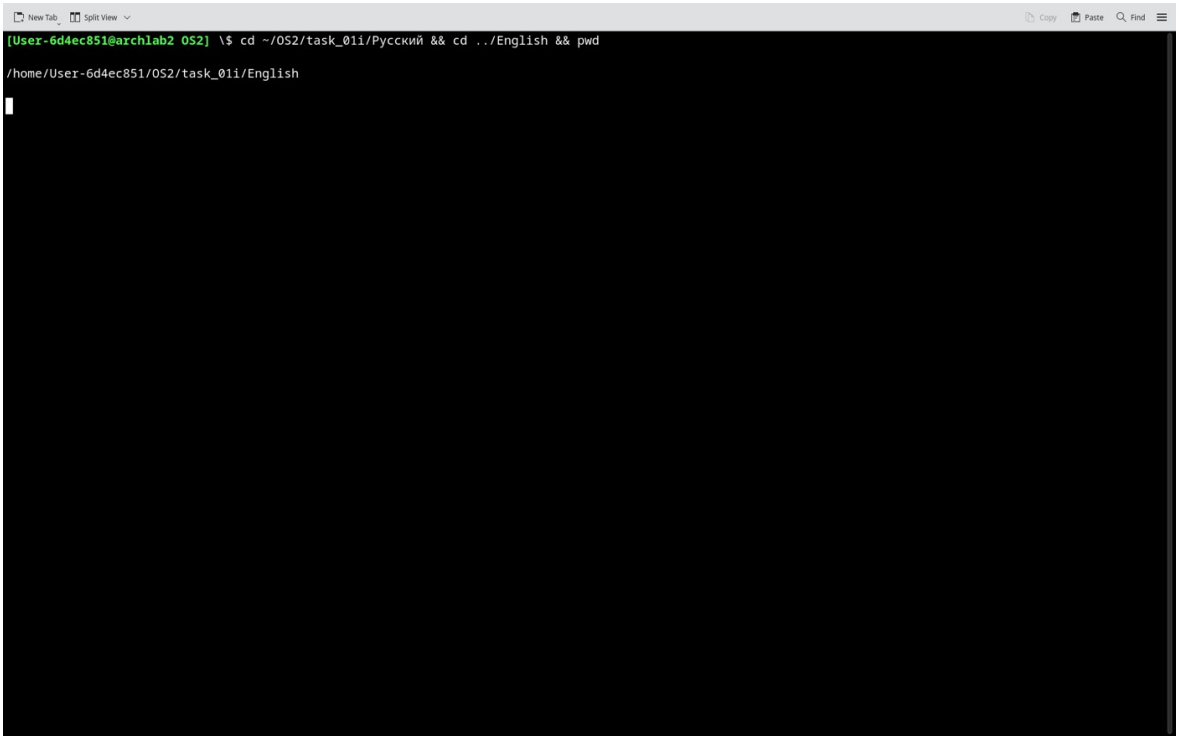


Рисунок 2 - Л4-5 п.1.2, аналог команды cd

4.1.3 Создание каталога task_01j в рабочем каталоге OS2

В соответствии с заданием Л4-5 п.1.3 выполняется команда:

```
mkdir -p ~/OS2/task_01j && ls -d ~/OS2/task_01j
```

Соответствующая команда Windows: md

Вывод команды:

```
/home/User-6d4ec851/OS2/task_01j
```



Рисунок 3 - Л4-5 п.1.3, аналог команды md

4.1.4 Создание иерархии каталогов task_01j/sub/dir/full/path

В соответствии с заданием Л4-5 п.1.4 выполняется команда:

```
mkdir -p ~/OS2/task_01j/sub/dir/full/path && find ~/OS2/task_01j -type d
```

Соответствующая команда Windows: md

Вывод команды:

```
/home/User-6d4ec851/OS2/task_01j
/home/User-6d4ec851/OS2/task_01j/sub
/home/User-6d4ec851/OS2/task_01j/sub/dir
/home/User-6d4ec851/OS2/task_01j/sub/dir/full
/home/User-6d4ec851/OS2/task_01j/sub/dir/full/path
```

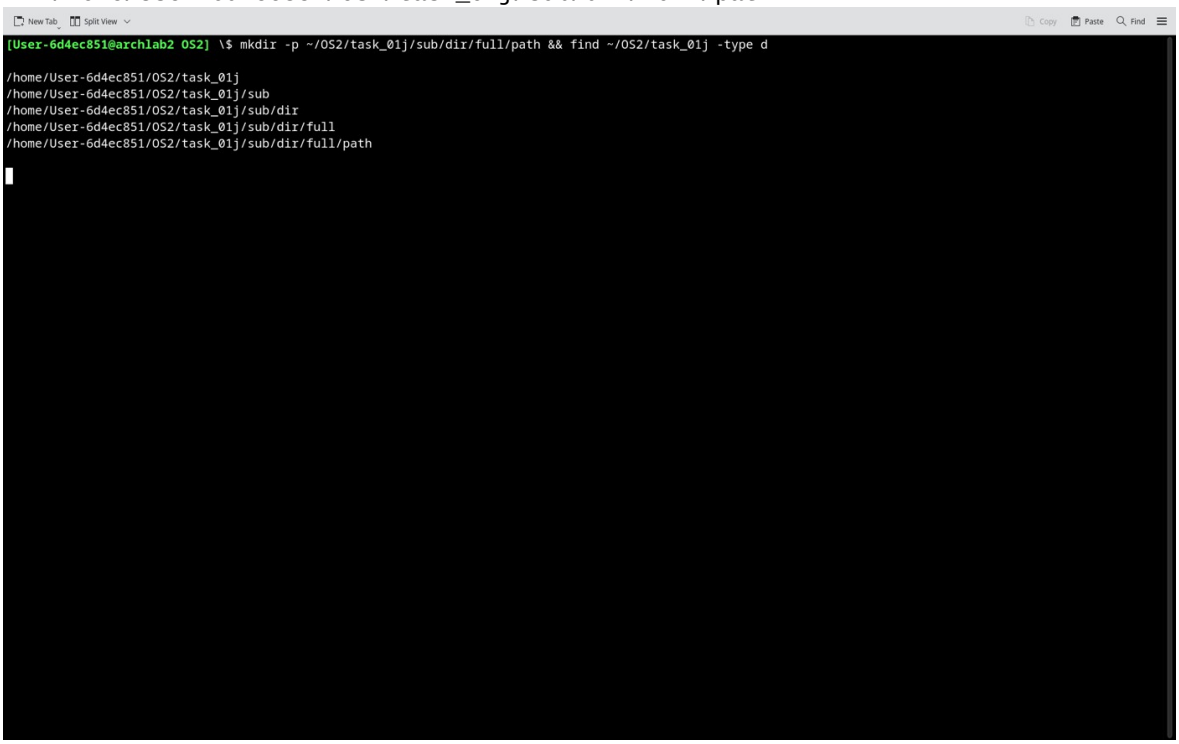


Рисунок 4 - Л4-5 п.1.4, аналог команды md

4.1.5 Удаление пустого каталога empty_subdir

В соответствии с заданием Л4-5 п.1.5 выполняется команда:

```
rmdir ~/OS2/task_01k/empty_subdir && ls ~/OS2/task_01k
```

Соответствующая команда Windows: rd

Вывод команды:

```
subdir
```

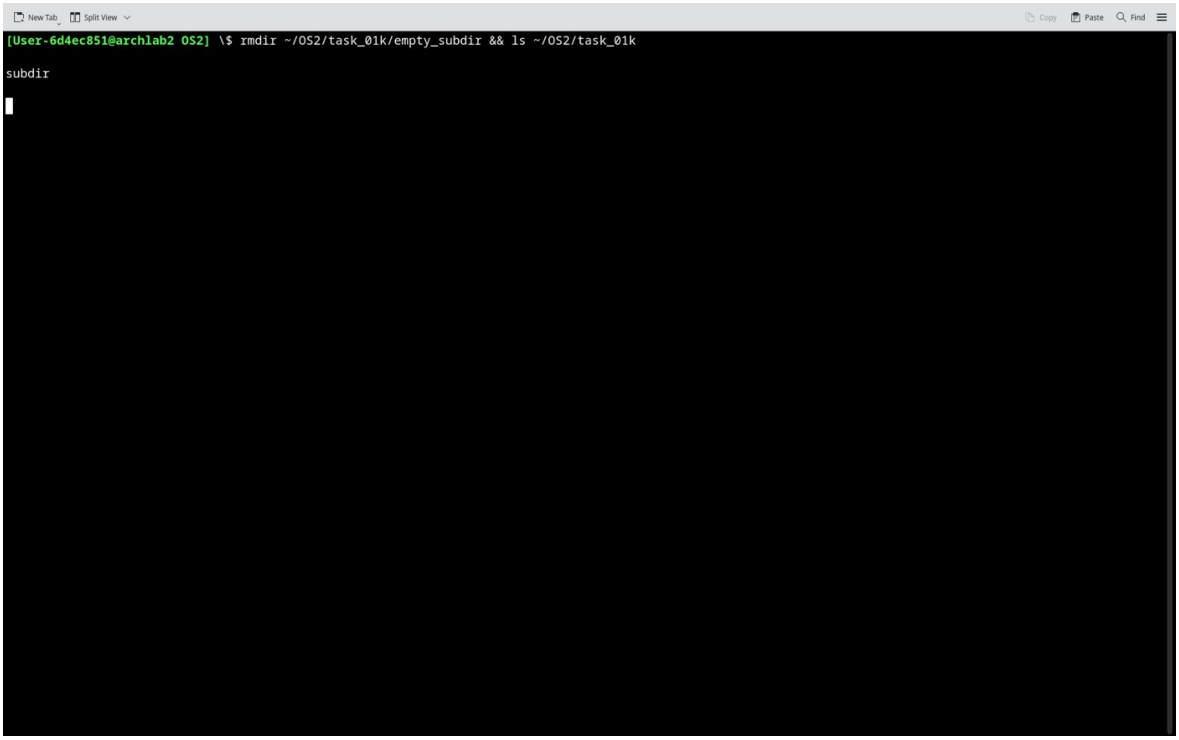


Рисунок 5 - Л4-5 п.1.5, аналог команды rd

4.2 Копирование и перемещение файлов

4.2.6 Копирование всех файлов txt из tasks_tmp в task_01a

В соответствии с заданием Л4-5 п.2.1 выполняется команда:

```
cp ~/OS2/tasks_tmp/*.txt ~/OS2/task_01a/ && ls ~/OS2/task_01a
```

Соответствующая команда Windows: copy

Вывод команды:

```
hidden1.txt
system1.txt
txt1.txt
txt2.txt
txt3.txt
```

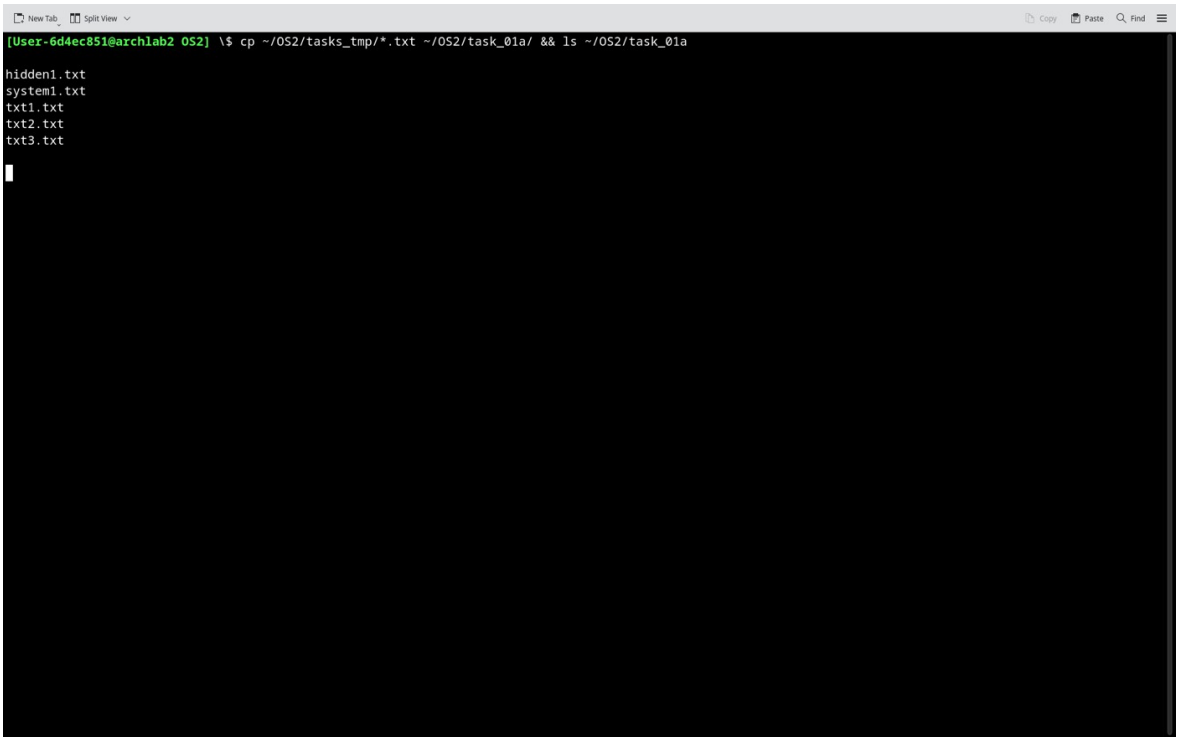


Рисунок 6 - Л4-5 п.2.1, аналог команды copy

4.2.7 Повторное копирование с запросом подтверждения перезаписи

В соответствии с заданием Л4-5 п.2.2 выполняется команда:

```
cp -iv ~/OS2/tasks_tmp/*.txt ~/OS2/task_01a/
```

Соответствующая команда Windows: copy

Данные, вводимые с консоли:

```
cp:   overwrite   '/home/User-6d4ec851/OS2/task_01a/hidden1.txt'?   cp:   overwrite
'/home/User-6d4ec851/OS2/task_01a/system1.txt'?               cp:               overwrite
'/home/User-6d4ec851/OS2/task_01a/txt1.txt'?                cp:                overwrite
'/home/User-6d4ec851/OS2/task_01a/txt2.txt'?                cp:                overwrite
'/home/User-6d4ec851/OS2/task_01a/txt3.txt'?
'/home/User-6d4ec851/OS2/tasks_tmp/hidden1.txt'                                        ->
'/home/User-6d4ec851/OS2/task_01a/hidden1.txt'
      '/home/User-6d4ec851/OS2/tasks_tmp/system1.txt'                                  ->
'/home/User-6d4ec851/OS2/task_01a/system1.txt'
      '/home/User-6d4ec851/OS2/tasks_tmp/txt1.txt'                                     ->
'/home/User-6d4ec851/OS2/task_01a/txt1.txt'
      '/home/User-6d4ec851/OS2/tasks_tmp/txt2.txt'                                     ->
'/home/User-6d4ec851/OS2/task_01a/txt2.txt'
```

```
'/home/User-6d4ec851/OS2/tasks_tmp/txt3.txt' ->
'/home/User-6d4ec851/OS2/task_01a/txt3.txt'
```

Вывод команды:

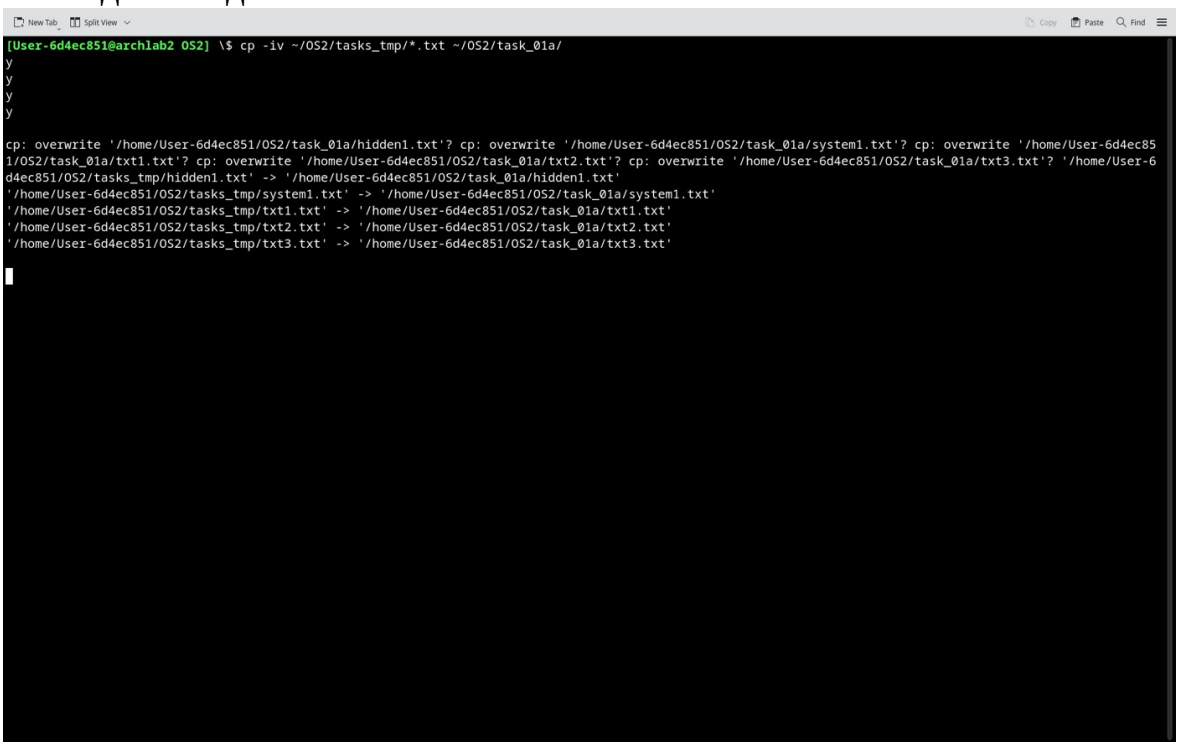


Рисунок 7 - Л4-5 п.2.2, аналог команды copy

4.2.8 Копирование файла в пределах task_01a с новым именем

В соответствии с заданием Л4-5 п.2.3 выполняется команда:

```
cp ~/OS2/task_01a/txt1.txt ~/OS2/task_01a/newname.txt && ls ~/OS2/task_01a
```

Соответствующая команда Windows: copy

Вывод команды:

```
hidden1.txt
newname.txt
system1.txt
txt1.txt
txt2.txt
txt3.txt
```

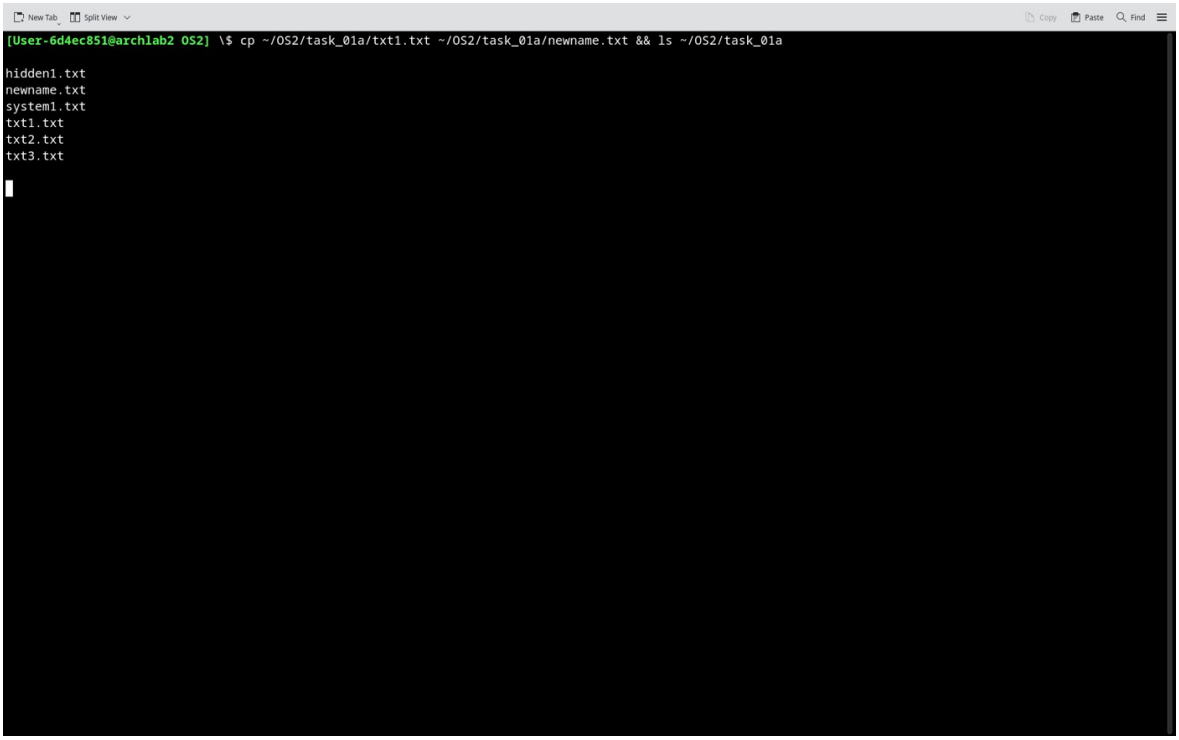


Рисунок 8 - Л4-5 п.2.3, аналог команды copy

4.2.9 Объединение файлов txt1.txt и txt2.txt в merged.txt

В соответствии с заданием Л4-5 п.2.4 выполняется команда:

```
cat ~/OS2/task_01a/txt1.txt ~/OS2/task_01a/txt2.txt > ~/OS2/task_01a/merged.txt && cat ~/OS2/task_01a/merged.txt
```

Соответствующая команда Windows: copy

Вывод команды:

```
txt1
txt2
```

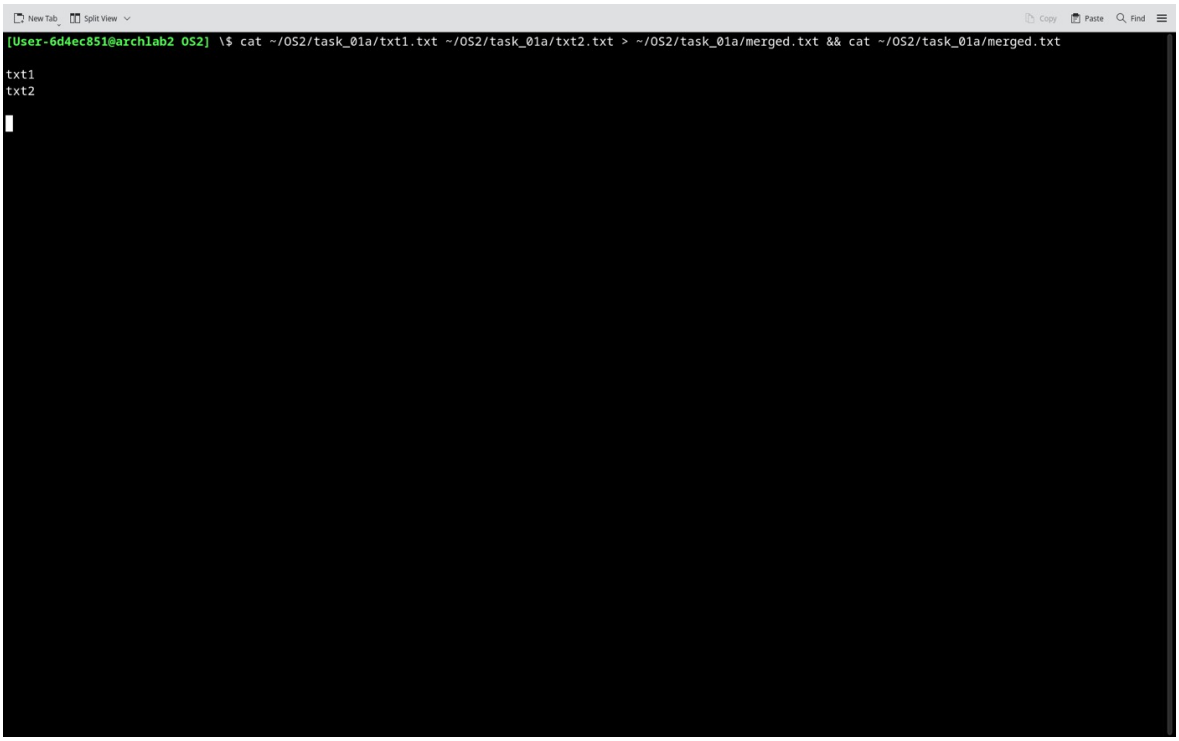


Рисунок 9 - Л4-5 п.2.4, аналог команды copy

4.2.10 Перемещение файлов txt из task_01c_src в task_01c_dst

В соответствии с заданием Л4-5 п.2.5 выполняется команда:

```
mv ~/OS2/task_01c_src/*.txt ~/OS2/task_01c_dst/ && ls ~/OS2/task_01c_dst
```

Соответствующая команда Windows: move

Вывод команды:

```
txt1.txt
txt2.txt
txt3.txt
txt4.txt
txt5.txt
```

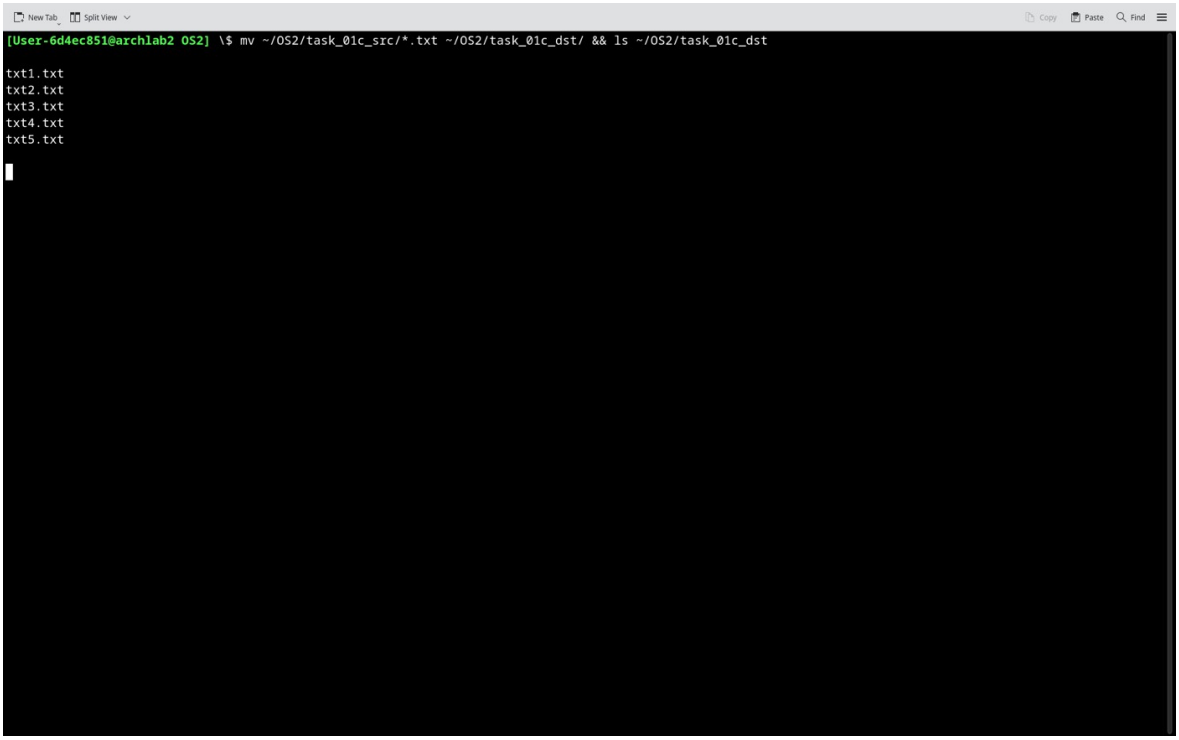


Рисунок 10 - Л4-5 п.2.5, аналог команды move

4.2.11 Изменение расширения всех файлов task_01e на bmp

В соответствии с заданием Л4-5 п.2.6 выполняется команда:

```
cd ~/OS2/task_01e && for f in *.txt; do mv «$f» «${f%.txt}.bmp»; done; ls ~/OS2/task_01e
```

Соответствующая команда Windows: ren

Вывод команды:

```
txt1.bmp
txt2.bmp
txt3.bmp
txt4.bmp
txt5.bmp
```

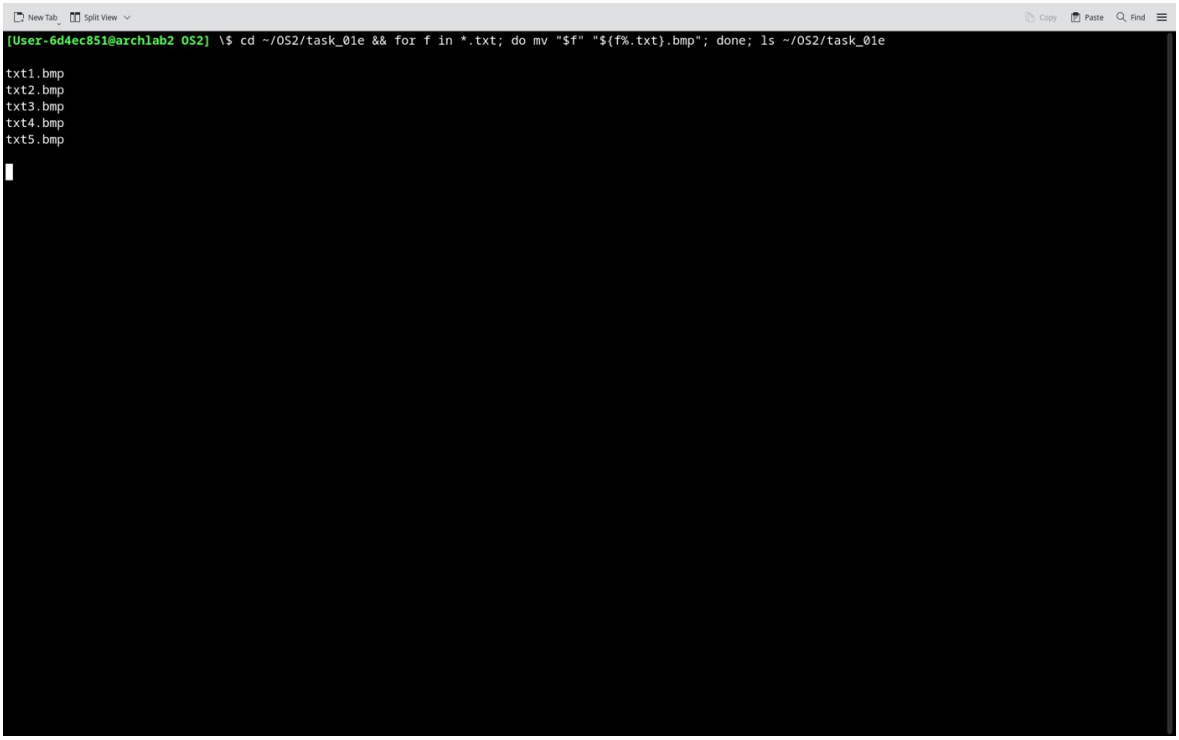


Рисунок 11 - Л4-5 п.2.6, аналог команды ren

4.2.12 Переименование txt1.bmp в файл с именем пользователя

В соответствии с заданием Л4-5 п.2.7 выполняется команда:

```
mv ~/OS2/task_01e/txt1.bmp ~/OS2/$USER.bmp && ls ~/OS2/*.bmp
```

Соответствующая команда Windows: ren

Вывод команды:

```
/home/User-6d4ec851/OS2/User-6d4ec851.bmp
```



Рисунок 12 - Л4-5 п.2.7, аналог команды ren

4.3 Просмотр и удаление файлов

4.3.13 Сравнение двух файлов каталога task_01f с записью в result.txt

В соответствии с заданием Л4-5 п.3.1 выполняется команда:

```
diff ~/OS2/task_01f/txt1.txt ~/OS2/task_01f/txt2.txt > ~/OS2/task_01f/result.txt; cat ~/OS2/task_01f/result.txt
```

Соответствующая команда Windows: fc

Вывод команды:

```
2c2
< Hello World
---
> Hello, World!
```

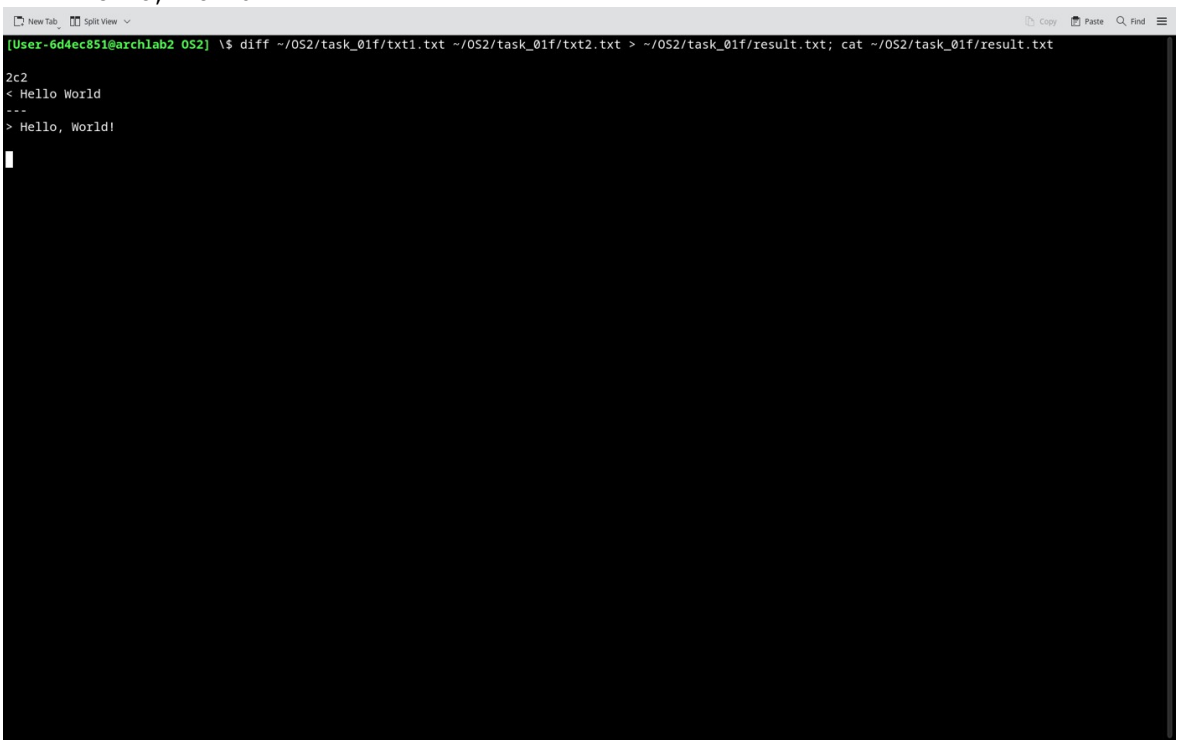


Рисунок 13 - Л4-5 п.3.1, аналог команды fc

4.3.14 Удаление файла txt1 каталога task_01g с подтверждением

В соответствии с заданием Л4-5 п.3.2 выполняется команда:

```
rm -iv ~/OS2/task_01g/txt1.txt
```

Соответствующая команда Windows: del

Данные, вводимые с консоли:

```
rm: remove regular file '/home/User-6d4ec851/OS2/task_01g/txt1.txt'? removed '/home/User-6d4ec851/OS2/task_01g/txt1.txt'
```

Вывод команды:

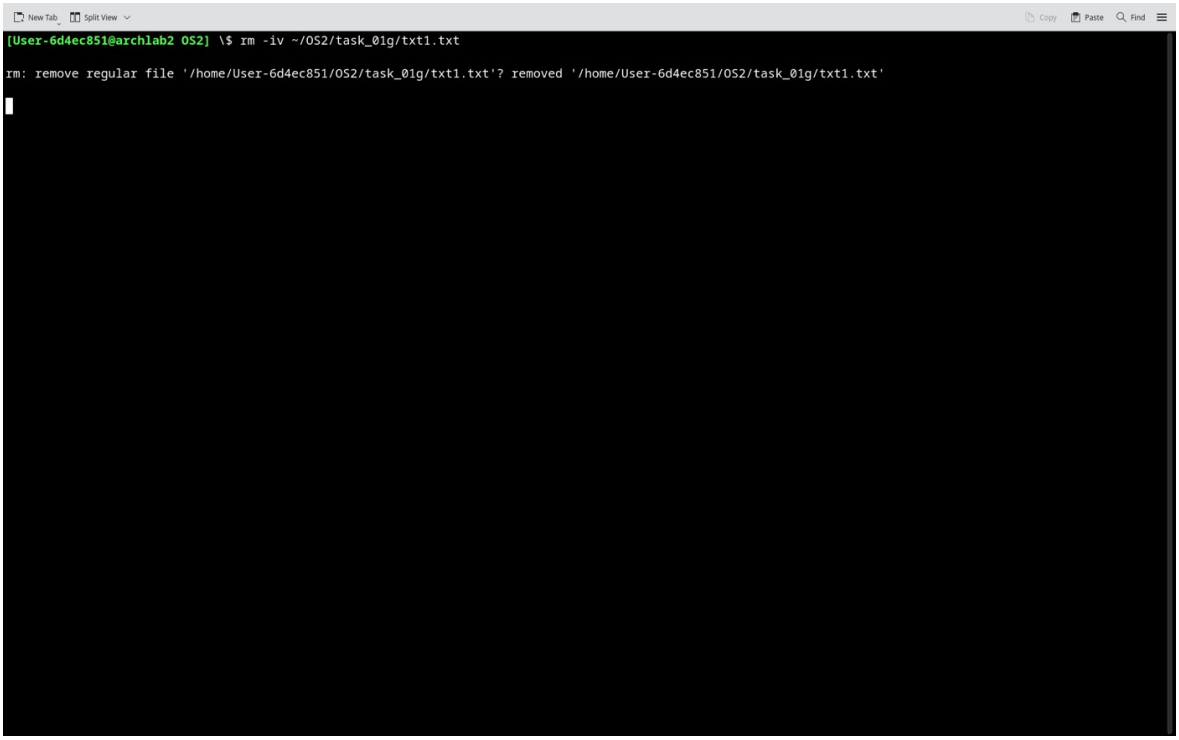


Рисунок 14 - Л4-5 п.3.2, аналог команды del

4.3.15 Удаление файлов с признаком системного из task_01g

В соответствии с заданием Л4-5 п.3.3 выполняется команда:

```
find ~/OS2/task_01g -maxdepth 1 -name 'system*' -type f -delete; ls ~/OS2/task_01g
```

Соответствующая команда Windows: del

Вывод команды:

```
hidden_dir
plain.txt
readonly.txt
subdir
```

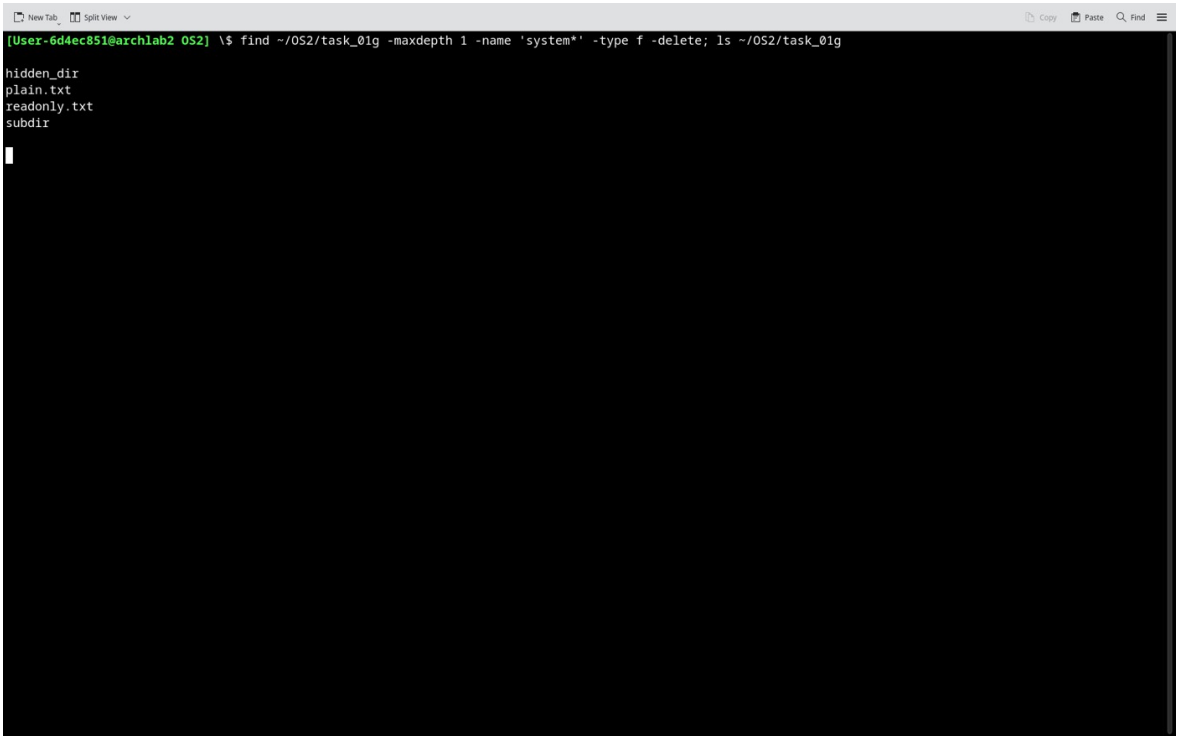


Рисунок 15 - Л4-5 п.3.3, аналог команды del

4.3.16 Удаление всех файлов task_01g, кроме имеющих режим только для чтения

В соответствии с заданием Л4-5 п.3.4 выполняется команда:

```
find ~/OS2/task_01g -perm -u=w -type f -delete; ls -l ~/OS2/task_01g
```

Соответствующая команда Windows: del

Вывод команды:

```
total 12
drwxr-xr-x 4 User-6d4ec851 User-6d4ec851 4096 Oct  4 22:10 hidden_dir
-r--r--r-- 1 User-6d4ec851 User-6d4ec851    9 Oct  4 22:09 readonly.txt
drwxr-xr-x 3 User-6d4ec851 User-6d4ec851 4096 Oct  4 22:10 subdir
```

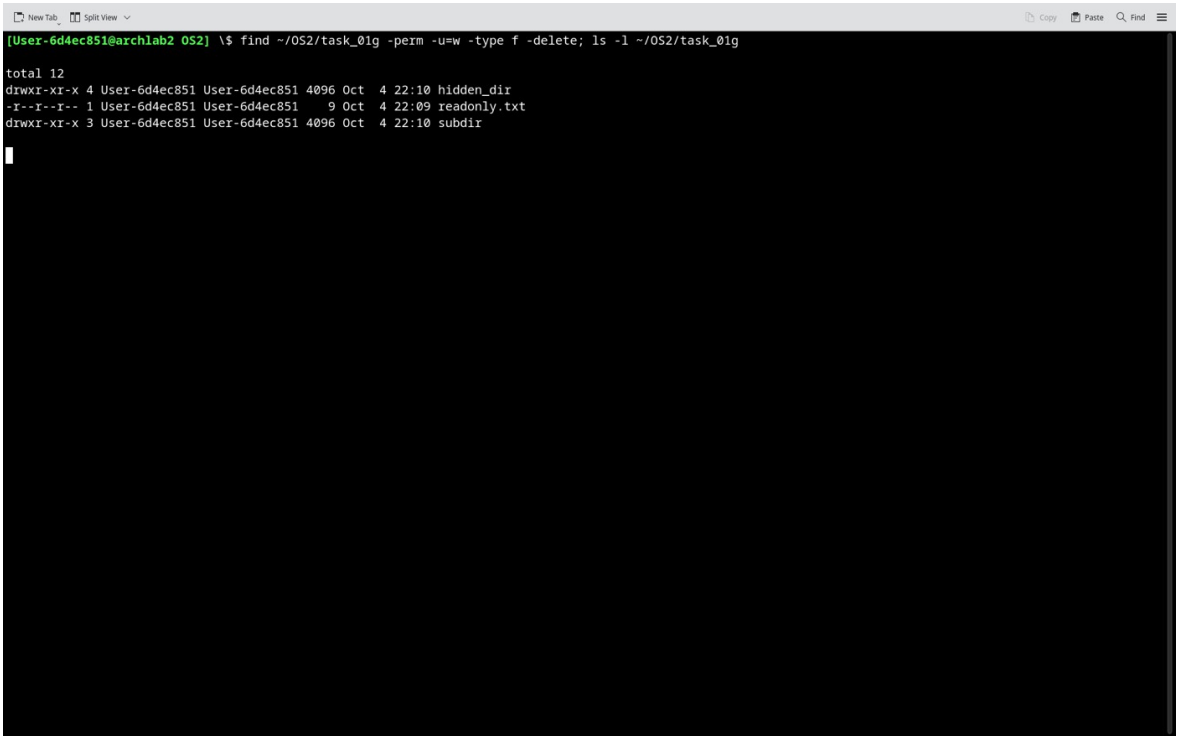


Рисунок 16 - Л4-5 п.3.4, аналог команды del

4.3.17 Удаление оставшихся файлов и дочерних каталогов task_01g

В соответствии с заданием Л4-5 п.3.5 выполняется команда:

```
rm -rf ~/OS2/task_01g/* && rmdir ~/OS2/task_01g && ls ~/OS2
```

Соответствующая команда Windows: del

Вывод команды:

```
User-6d4ec851.bmp
bat_01.sh
bat_02.sh
bat_03.sh
bat_04.sh
bat_05.sh
calc.sh
path_count.sh
prep.sh
task_01a
task_01b
task_01c_dst
task_01c_src
task_01d_1
task_01d_2
task_01e
task_01f
task_01h
task_01i
task_01j
task_01k
tasks_tmp
tempconv.sh
top5.sh
tree_sizes.sh
```

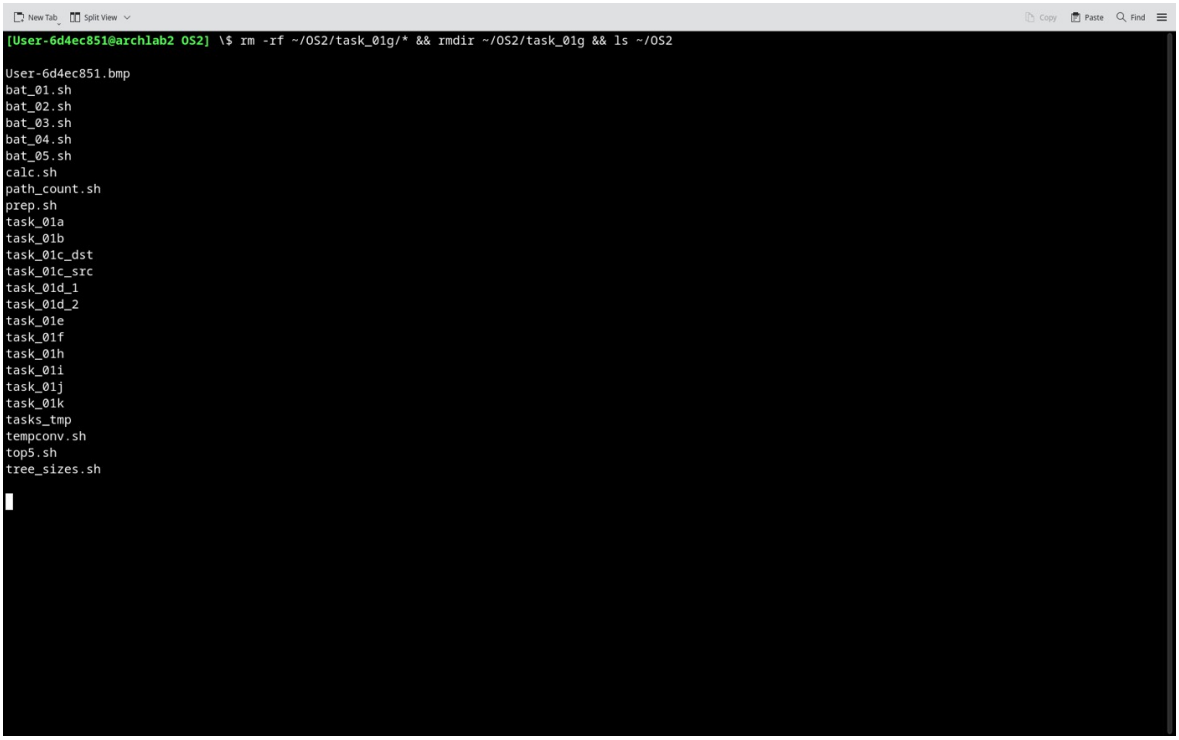


Рисунок 17 - Л4-5 п.3.5, аналог команды del

4.3.18 Рекурсивный вывод содержимого рабочего каталога

В соответствии с заданием Л4-5 п.3.6 выполняется команда:

```
ls -R ~/OS2 | head -28
```

Соответствующая команда Windows: dir

Вывод команды:

```
/home/User-6d4ec851/OS2:
User-6d4ec851.bmp
bat_01.sh
bat_02.sh
bat_03.sh
bat_04.sh
bat_05.sh
calc.sh
path_count.sh
prep.sh
task_01a
task_01b
task_01c_dst
task_01c_src
task_01d_1
task_01d_2
task_01e
task_01f
task_01h
task_01i
task_01j
task_01k
tasks_tmp
tempconv.sh
top5.sh
tree_sizes.sh
... ещё 1 строк вывода
```

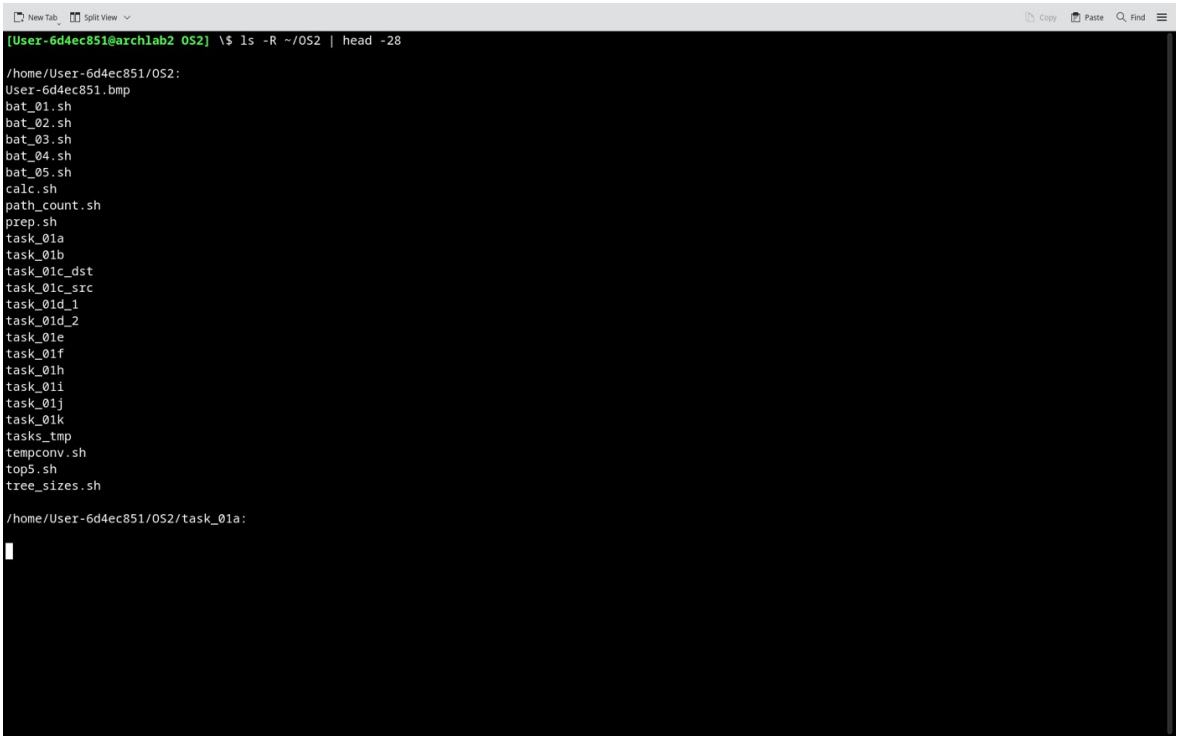


Рисунок 18 - Л4-5 п.3.6, аналог команды dir

4.3.19 Вывод списка файлов в алфавитном порядке с записью в user_files.txt

В соответствии с заданием Л4-5 п.3.7 выполняется команда:

```
ls -1 ~/OS2 > ~/OS2/task_01h/user_files.txt && cat ~/OS2/task_01h/user_files.txt
```

Соответствующая команда Windows: dir

Вывод команды:

```
User-6d4ec851.bmp
bat_01.sh
bat_02.sh
bat_03.sh
bat_04.sh
bat_05.sh
calc.sh
path_count.sh
prep.sh
task_01a
task_01b
task_01c_dst
task_01c_src
task_01d_1
task_01d_2
task_01e
task_01f
task_01h
task_01i
task_01j
task_01k
tasks_tmp
tempconv.sh
top5.sh
tree_sizes.sh
```

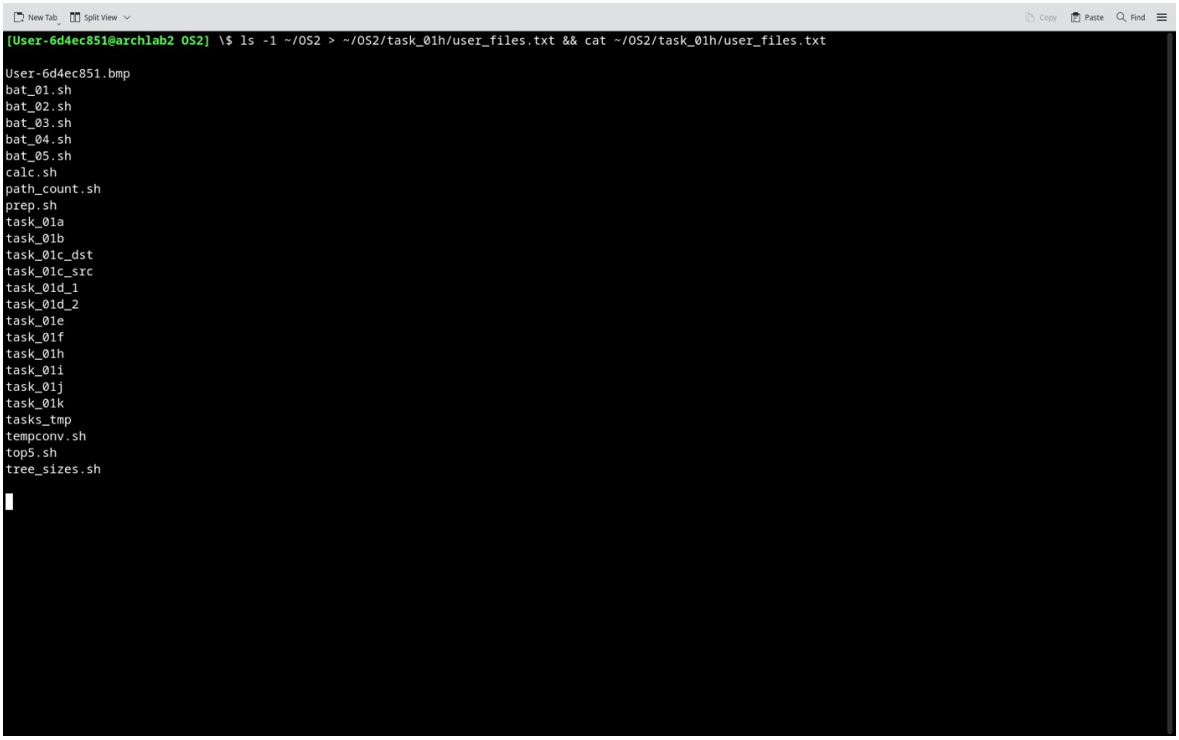


Рисунок 19 - Л4-5 п.3.7, аналог команды dir

4.4 Перенаправление вывода и конвейеры

4.4.20 Создание файла со списком студентов вводом с консоли

В соответствии с заданием Л4-5 п.4.2 выполняется команда:

```
cat > ~/OS2/students.txt
```

Соответствующая команда Windows: type

Вывод команды:

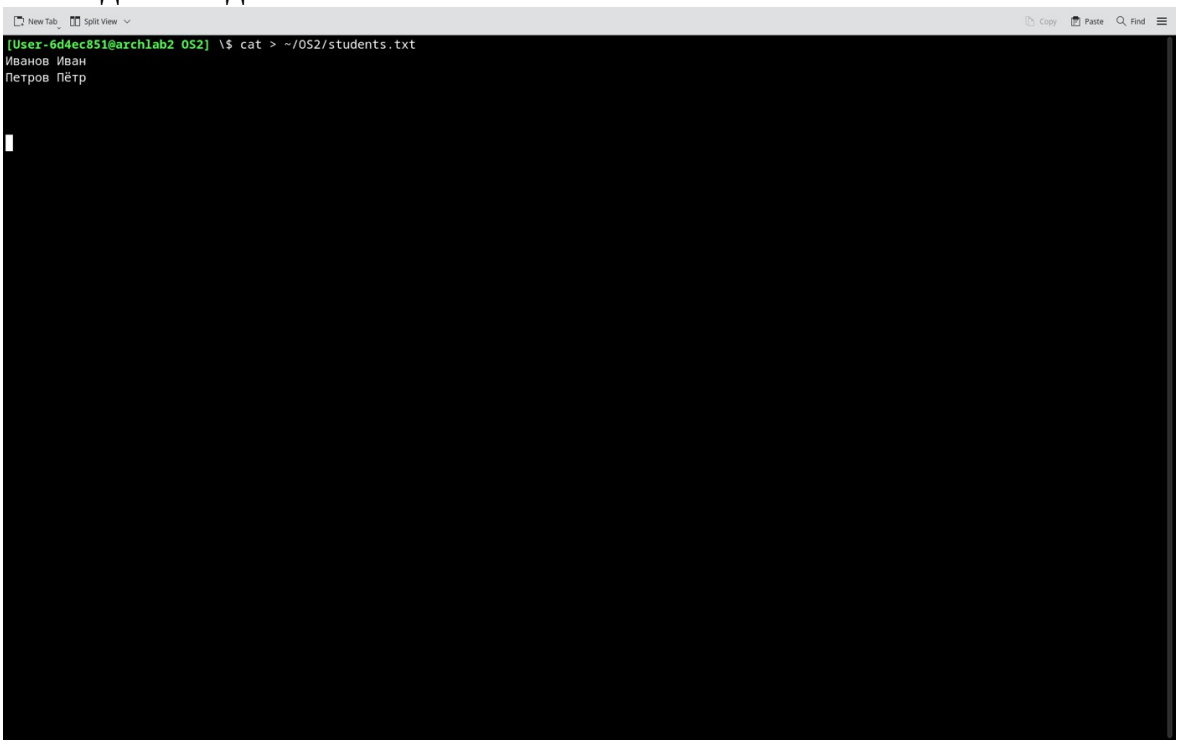


Рисунок 20 - Л4-5 п.4.2, аналог команды type

4.4.21 Поиск студента по фамилии в списке

В соответствии с заданием Л4-5 п.4.3 выполняется команда:

```
grep «Иванов» ~/OS2/students.txt
```

Соответствующая команда Windows: find

Вывод команды:

```
Иванов Иван
```

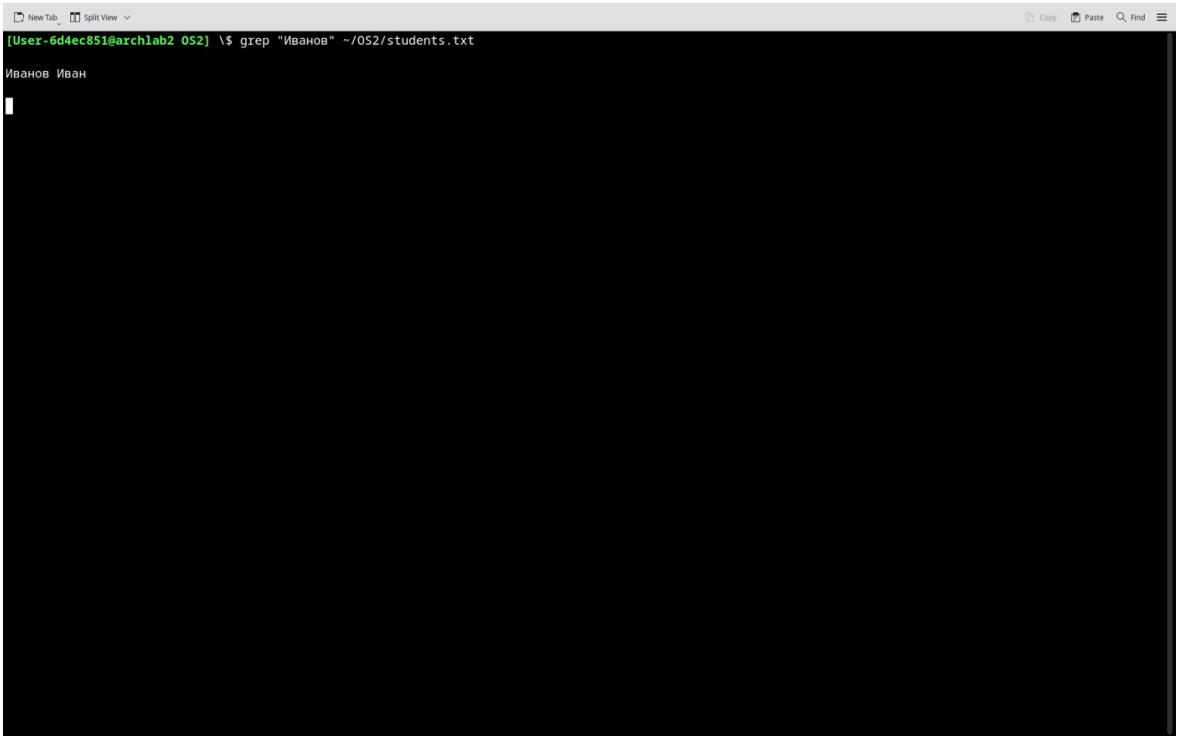


Рисунок 21 - Л4-5 п.4.3, аналог команды find

4.4.22 Добавление студентов в конец файла

В соответствии с заданием Л4-5 п.4.4 выполняется команда:

```
echo «Кузнецов Кузьма» >> ~/OS2/students.txt && cat ~/OS2/students.txt
```

Соответствующая команда Windows: echo

Вывод команды:

```
Иванов Иван
Петров Пётр
Сидоров Сидор
Кузнецов Кузьма
```

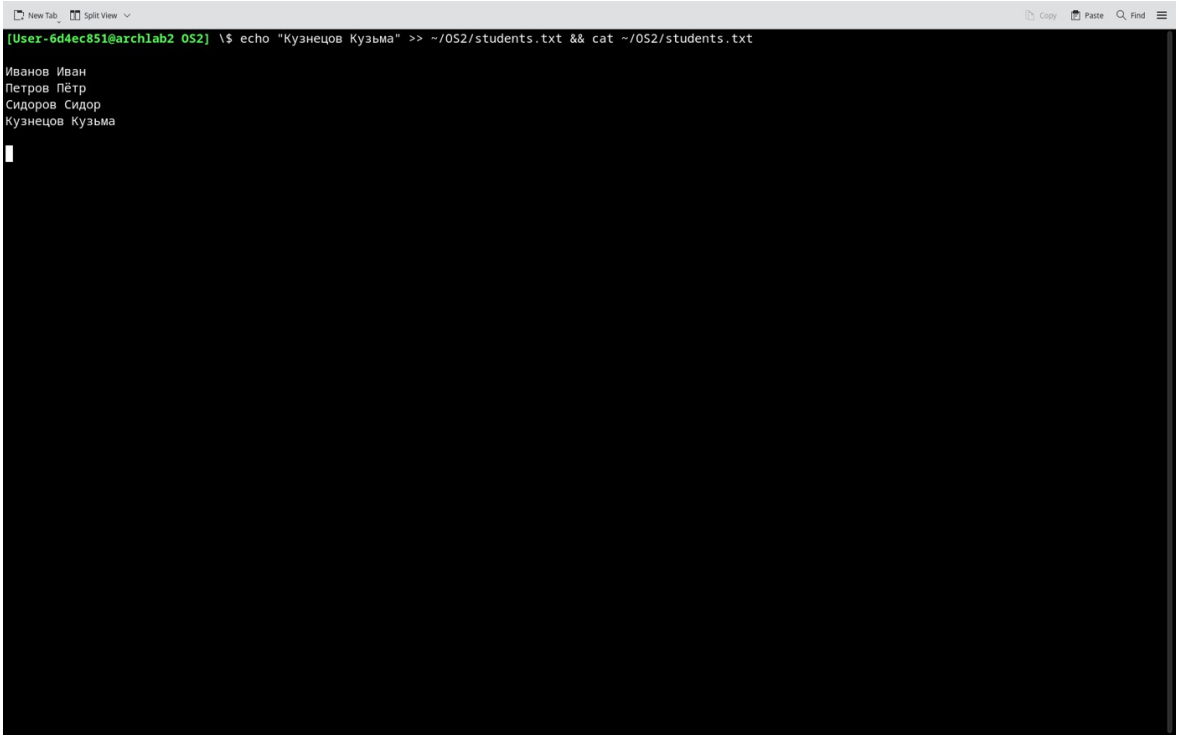


Рисунок 22 - Л4-5 п.4.4, аналог команды echo

4.4.23 Вывод списка студентов в отсортированном порядке

В соответствии с заданием Л4-5 п.4.5 выполняется команда:

```
sort ~/OS2/students.txt
```

Соответствующая команда Windows: sort

Вывод команды:

```
Иванов Иван
Кузнецов Кузьма
Петров Пётр
Сидоров Сидор
```

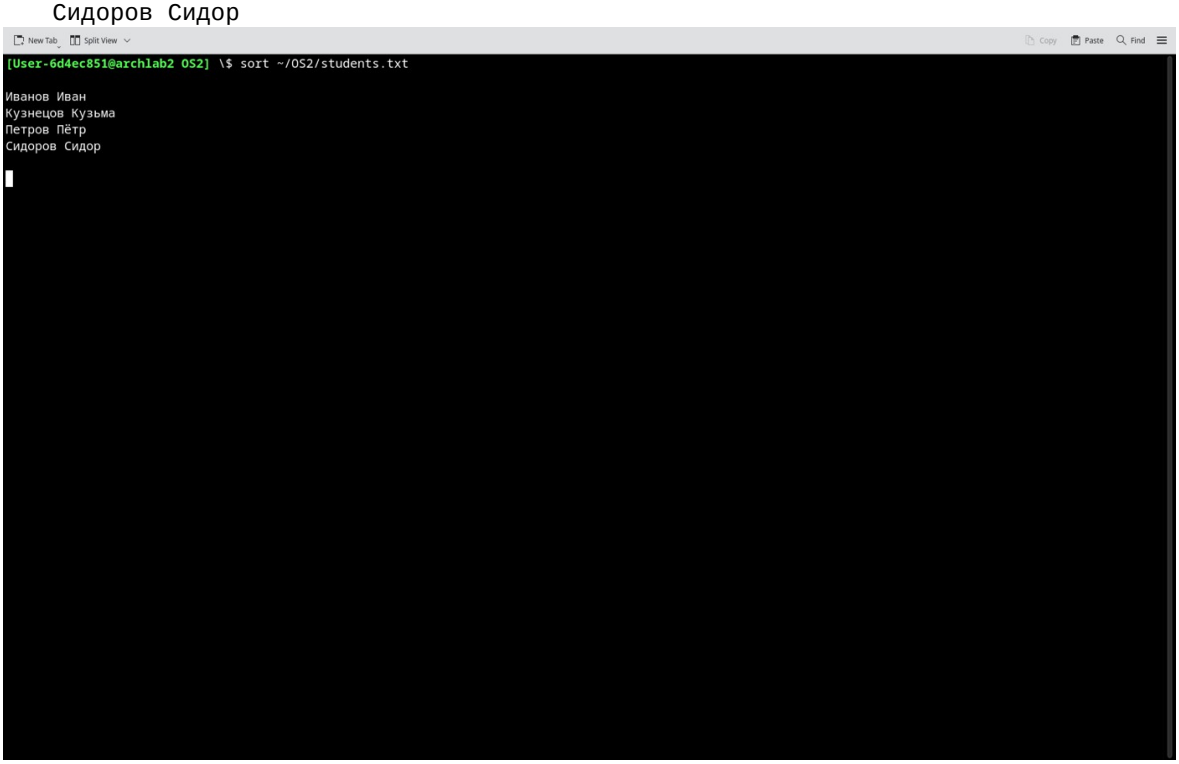


Рисунок 23 - Л4-5 п.4.5, аналог команды sort

4.4.24 Получение версии BIOS через промежуточный файл

В соответствии с заданием Л4-5 п.4.6 выполняется команда:

```
cat /sys/class/dmi/id/bios_version > ~/OS2/bios.txt; cat ~/OS2/bios.txt
```

Соответствующая команда Windows: systeminfo

Вывод команды:

```
VMW201.00V.21805430.BA64.2305221830
```

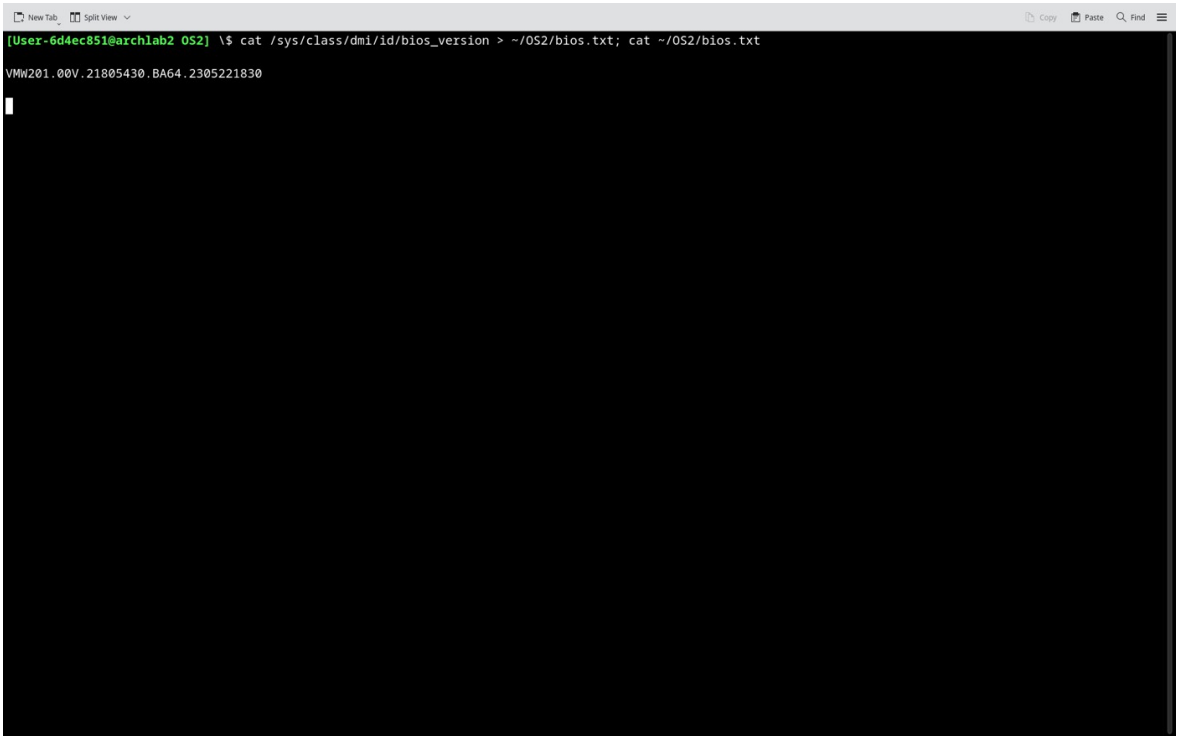


Рисунок 24 - Л4-5 п.4.6, аналог команды systeminfo

4.5 Простейшие сценарии и переменные окружения

4.5.25 Права доступа к сценариям: ключ +x

В соответствии с заданием Л4-5 п.5 (Linux) выполняется команда:

```
cd ~/OS2 && chmod +x *.sh && ls -l ~/OS2/*.sh
```

Соответствующая команда Windows: attrib

Вывод команды:

```
-rwxr-xr-x    1    User-6d4ec851    User-6d4ec851        354    Oct        4    21:46
/home/User-6d4ec851/OS2/bat_01.sh
-rwxr-xr-x    1    User-6d4ec851    User-6d4ec851        451    Oct        4    21:46
/home/User-6d4ec851/OS2/bat_02.sh
-rwxr-xr-x    1    User-6d4ec851    User-6d4ec851        390    Oct        4    21:46
/home/User-6d4ec851/OS2/bat_03.sh
-rwxr-xr-x    1    User-6d4ec851    User-6d4ec851        658    Oct        4    21:46
/home/User-6d4ec851/OS2/bat_04.sh
-rwxr-xr-x    1    User-6d4ec851    User-6d4ec851        878    Oct        4    21:46
/home/User-6d4ec851/OS2/bat_05.sh
-rwxr-xr-x    1    User-6d4ec851    User-6d4ec851    3430    Oct        4    19:55
/home/User-6d4ec851/OS2/calc.sh
-rwxr-xr-x    1    User-6d4ec851    User-6d4ec851    1527    Oct        4    19:55
/home/User-6d4ec851/OS2/path_count.sh
-rwxr-xr-x    1    User-6d4ec851    User-6d4ec851    5579    Oct        4    21:42
/home/User-6d4ec851/OS2/prep.sh
-rwxr-xr-x    1    User-6d4ec851    User-6d4ec851    1827    Oct        4    19:55
/home/User-6d4ec851/OS2/tempconv.sh
-rwxr-xr-x    1    User-6d4ec851    User-6d4ec851    2484    Oct        4    19:55
/home/User-6d4ec851/OS2/top5.sh
-rwxr-xr-x    1    User-6d4ec851    User-6d4ec851    2597    Oct        4    19:55
/home/User-6d4ec851/OS2/tree_sizes.sh
```

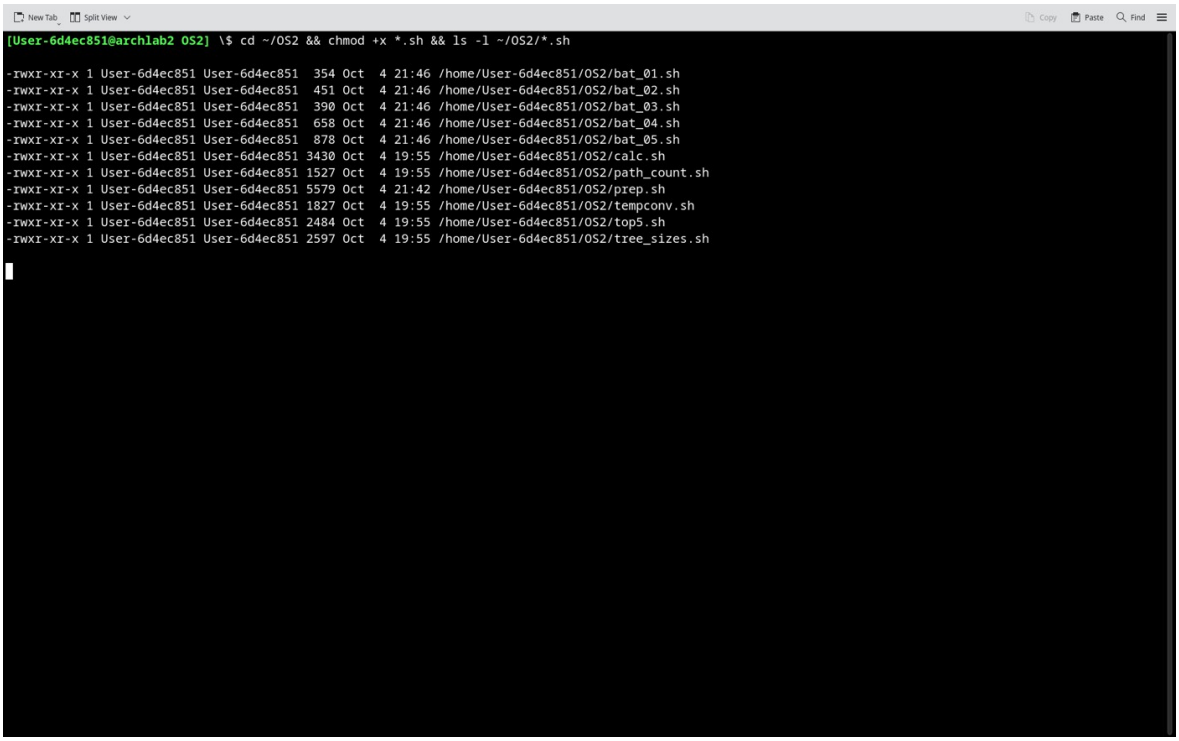


Рисунок 25 - Л4-5 п.5 (Linux), аналог команды attrib

4.5.26 Просмотр переменных окружения командой env

В соответствии с заданием Л4-5 п.5 (Linux) выполняется команда:

```
env | sort | head -14
```

Соответствующая команда Windows: set

Вывод команды:

```
COLORFGBG=15;0
COLORTERM=truecolor
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1001/bus
DISPLAY=:1
DOTNET_ROOT=/home/User-6d4ec851/.dotnet
FLATPAK_TTY_PROGRESS=1
HOME=/home/User-6d4ec851
KONSOLE_DBUS_ACTIVATION_COOKIE=7CFZ/yMcfDZ1KEa4QOg2ji5LbagqG70NxdcK2E+G3R4=
KONSOLE_DBUS_SERVICE=:1.1025
KONSOLE_DBUS_SESSION=/Sessions/1
KONSOLE_DBUS_WINDOW=/Windows/1
KONSOLE_VERSION=260801
LANG=C
LANGUAGE=
```

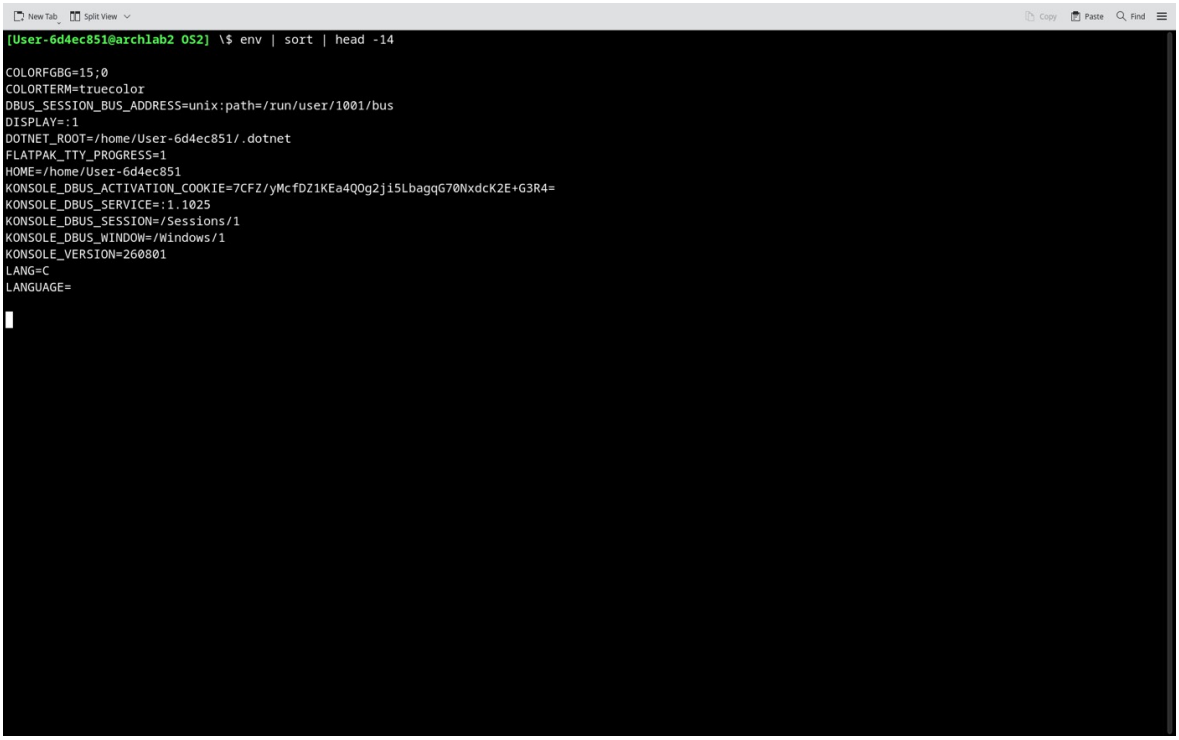


Рисунок 26 - Л4-5 п.5 (Linux), аналог команды set

Вывод по разделу 4

В разделе 4 выполнено 26 заданий. Каждое задание сопоставлено с командой Windows-части, зафиксировано снимком экрана и сохранено текстовым выводом. Выполнение подтверждено автоматизированной проверкой распознавания текста на снимках: во всех кадрах присутствуют строка с введённой командой и результат её выполнения.

Задания раздела 4 выполнены полностью, команды отработали без ошибок, кроме заданий, где ошибочная ситуация является частью проверяемого сценария.

5 ВЫПОЛНЕНИЕ ЗАДАНИЙ УРОВНЕЙ 6-7

Задания уровней 6-7 выполнены в консоли Arch Linux ARM. Рассмотрены расширенные операции копирования с сохранением атрибутов, работа с консольным вводом и выводом, сценарии с параметрами и исследование переменных окружения. Сценарии bat_01 - bat_05 из задания переписаны на bash с сохранением логики работы и добавлением обработки ошибочных ситуаций.

5.1 Расширенное копирование и атрибуты

5.1.1 Копирование всех файлов и каталогов, включая пустые, из task_01b

В соответствии с заданием Л6-7 п.1.1 выполняется команда:

```
cp -r ~/OS2/task_01b ~/OS2/task_01b_dst_1 && find ~/OS2/task_01b_dst_1 | sort
```

Соответствующая команда Windows: xcopy

Вывод команды:

```
/home/User-6d4ec851/OS2/task_01b_dst_1
/home/User-6d4ec851/OS2/task_01b_dst_1/archive-readonly.txt
/home/User-6d4ec851/OS2/task_01b_dst_1/hidden_dir
/home/User-6d4ec851/OS2/task_01b_dst_1/hidden_dir/empty_subdir
/home/User-6d4ec851/OS2/task_01b_dst_1/hidden_dir/subdir
/home/User-6d4ec851/OS2/task_01b_dst_1/hidden_dir/subdir/txt1.txt
/home/User-6d4ec851/OS2/task_01b_dst_1/hidden_dir/subdir/txt2.txt
/home/User-6d4ec851/OS2/task_01b_dst_1/hidden_dir/system1.txt
/home/User-6d4ec851/OS2/task_01b_dst_1/hidden_dir/txt1.txt
/home/User-6d4ec851/OS2/task_01b_dst_1/plain.txt
/home/User-6d4ec851/OS2/task_01b_dst_1/readonly.txt
/home/User-6d4ec851/OS2/task_01b_dst_1/subdir
/home/User-6d4ec851/OS2/task_01b_dst_1/subdir/empty_subdir
/home/User-6d4ec851/OS2/task_01b_dst_1/subdir/txt1.txt
/home/User-6d4ec851/OS2/task_01b_dst_1/subdir/txt2.txt
```

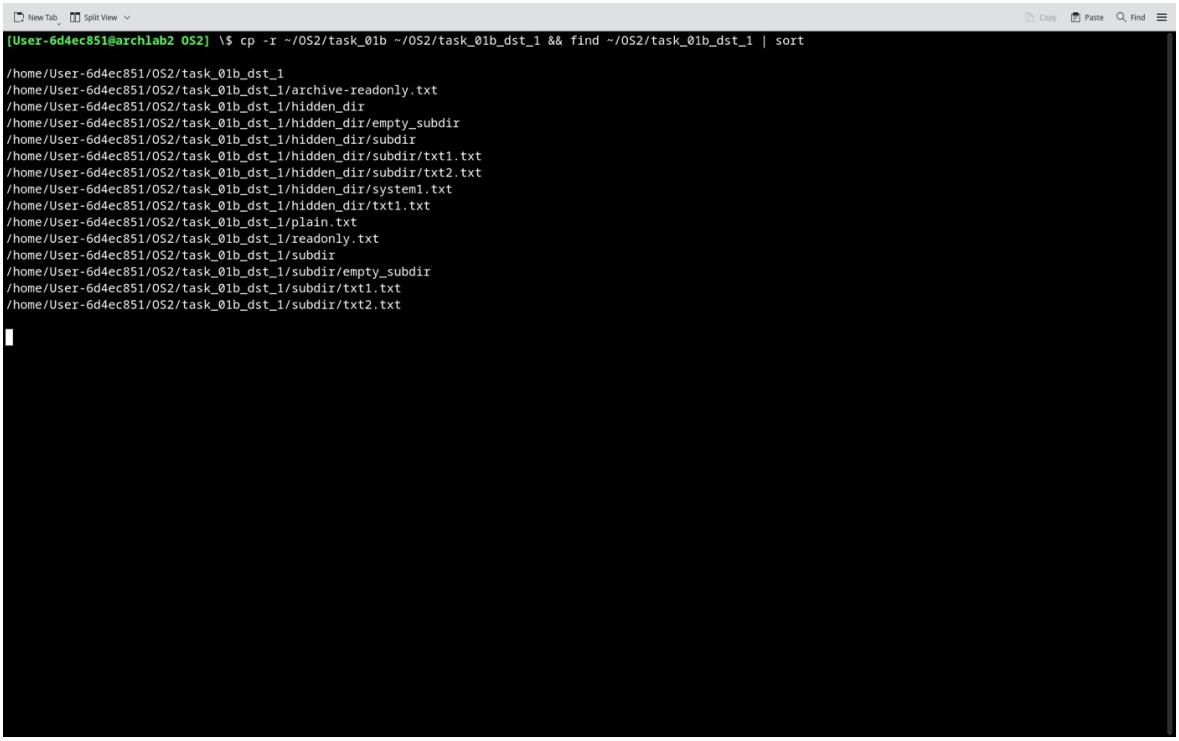


Рисунок 27 - Л6-7 п.1.1, аналог команды xcopy

5.1.2 Копирование файлов с сохранением атрибутов из task_01b

В соответствии с заданием Л6-7 п.1.2 выполняется команда:

```
cp -rp ~/OS2/task_01b ~/OS2/task_01b_dst_2 && ls -l ~/OS2/task_01b_dst_2
```

Соответствующая команда Windows: xcopy

Вывод команды:

```
total 20
-r--r--r-- 1 User-6d4ec851 User-6d4ec851    8 Oct  4 22:11 archive-readonly.txt
drwxr-xr-x 4 User-6d4ec851 User-6d4ec851 4096 Oct  4 22:11 hidden_dir
```

```
-rw-r--r-- 1 User-6d4ec851 User-6d4ec851    6 Oct  4 22:11 plain.txt
-r--r--r-- 1 User-6d4ec851 User-6d4ec851    9 Oct  4 22:11 readonly.txt
drwxr-xr-x 3 User-6d4ec851 User-6d4ec851 4096 Oct  4 22:11 subdir
```

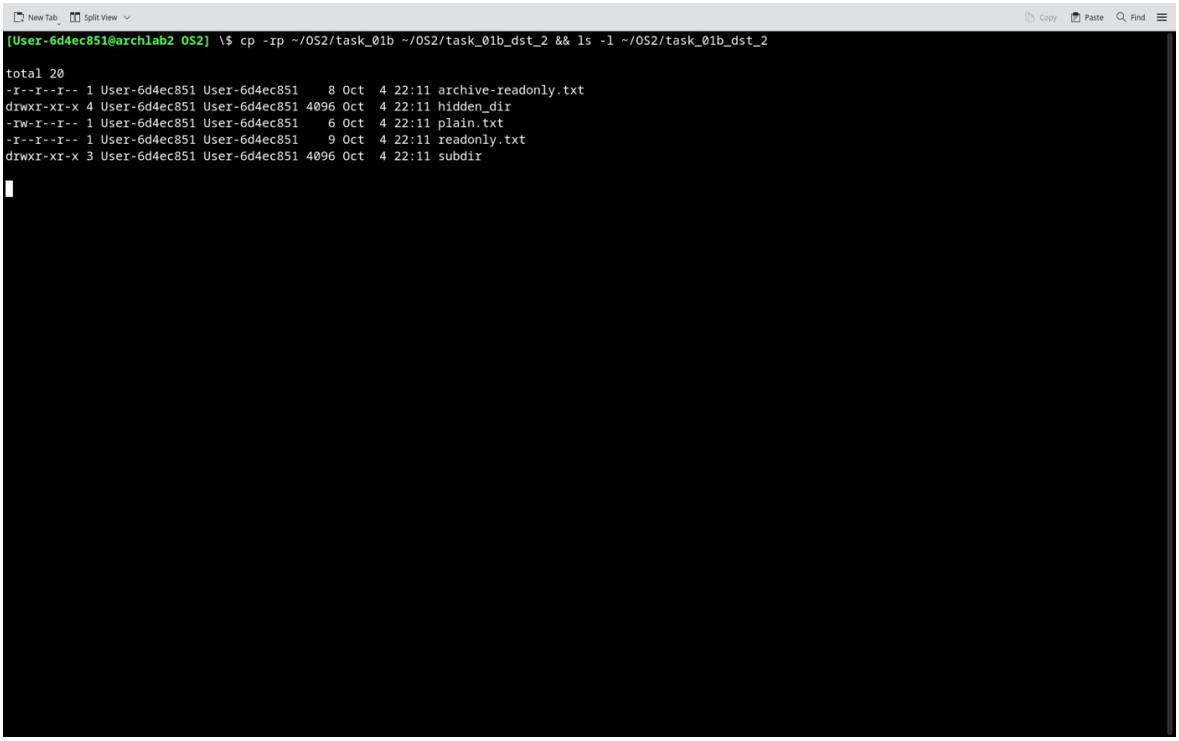


Рисунок 28 - Л6-7 п.1.2, аналог команды xcopy

5.1.3 Замена файлов в task_01d_2 файлами из task_01d_1

В соответствии с заданием Л6-7 п.1.3 выполняется команда:

```
cp -f ~/OS2/task_01d_1/* ~/OS2/task_01d_2/ && ls ~/OS2/task_01d_2
```

Соответствующая команда Windows: replace

Вывод команды:

```
txt1.txt
txt2.txt
txt3.txt
```

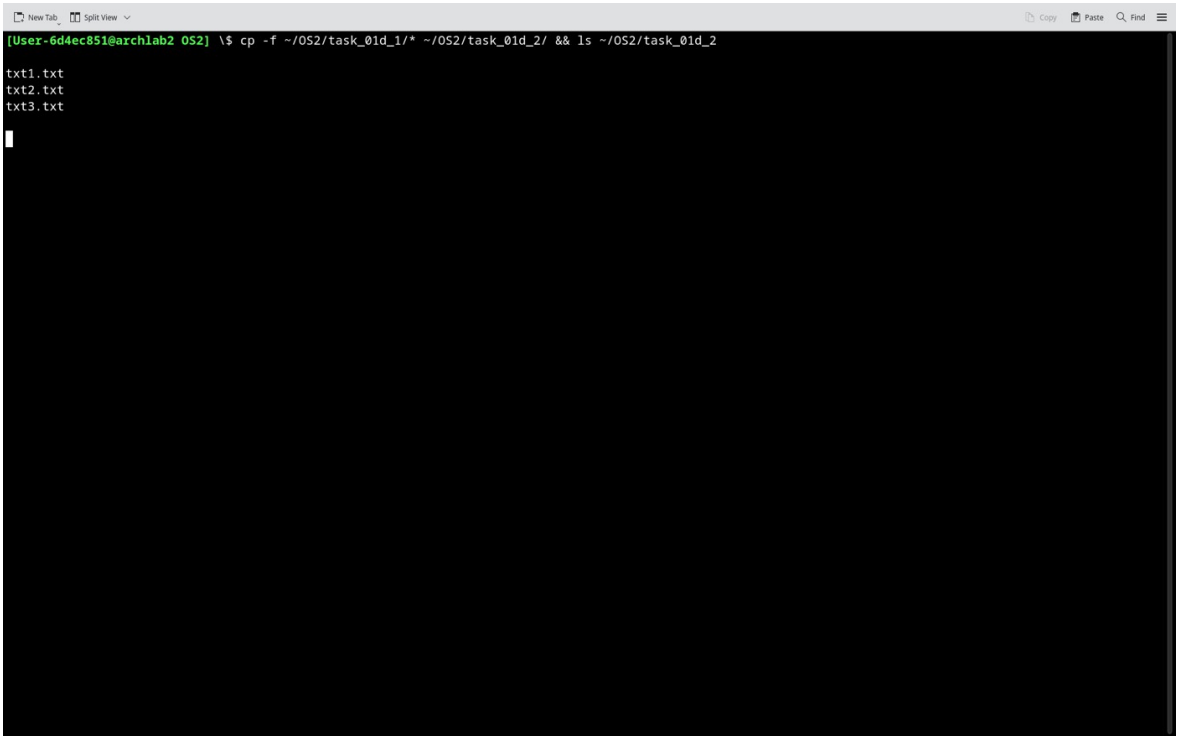


Рисунок 29 - Л6-7 п.1.3, аналог команды replace

5.1.4 Удаление текста из файлов каталога task_01d_2

В соответствии с заданием Л6-7 п.1.4 выполняется команда:

```
cd ~/OS2/task_01d_2 && for f in *.txt; do > «$f»; done; ls -l ~/OS2/task_01d_2
```

Соответствующая команда Windows: copy

Вывод команды:

```
total 0
-rw-r--r-- 1 User-6d4ec851 User-6d4ec851 0 Oct  4 22:11 txt1.txt
-rw-r--r-- 1 User-6d4ec851 User-6d4ec851 0 Oct  4 22:11 txt2.txt
-rw-r--r-- 1 User-6d4ec851 User-6d4ec851 0 Oct  4 22:11 txt3.txt
```

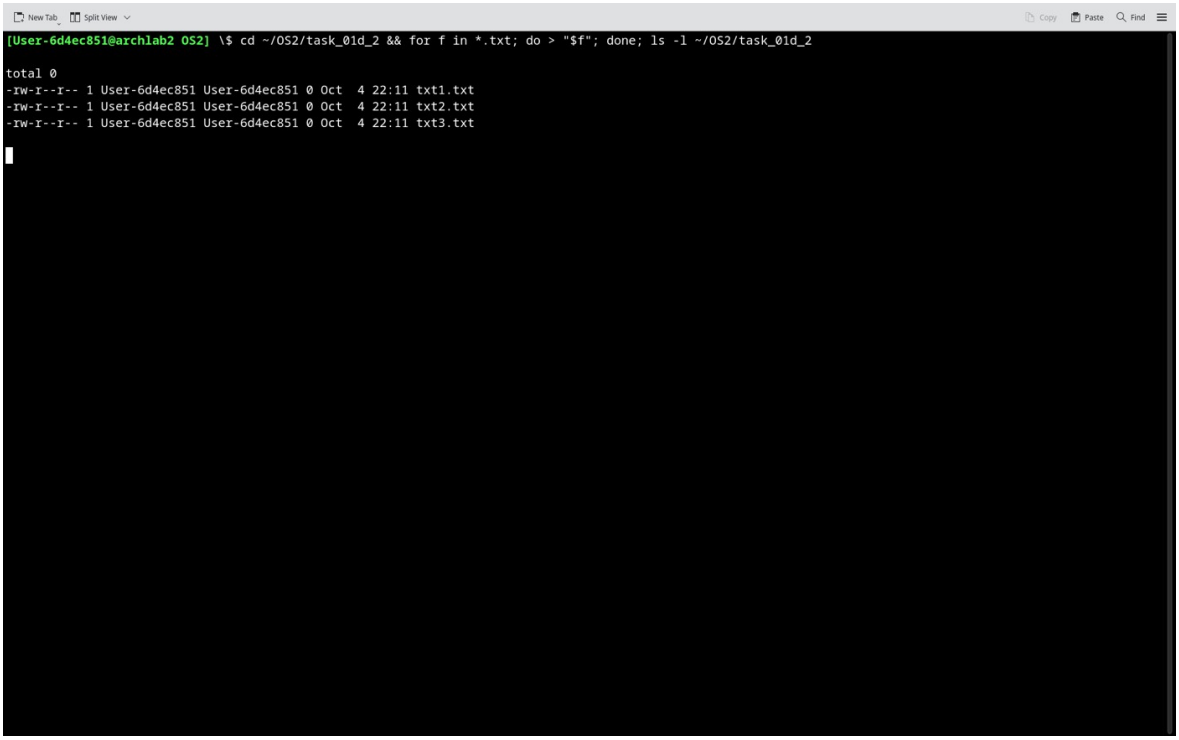


Рисунок 30 - Л6-7 п.1.4, аналог команды copy

5.1.5 Удаление непустого каталога task_01k/subdir

В соответствии с заданием Л6-7 п.1.5 выполняется команда:

```
rm -rf ~/OS2/task_01k/subdir && ls -R ~/OS2/task_01k
```

Соответствующая команда Windows: rd

Вывод команды:

```
/home/User-6d4ec851/OS2/task_01k:
empty_subdir
/home/User-6d4ec851/OS2/task_01k/empty_subdir:
```

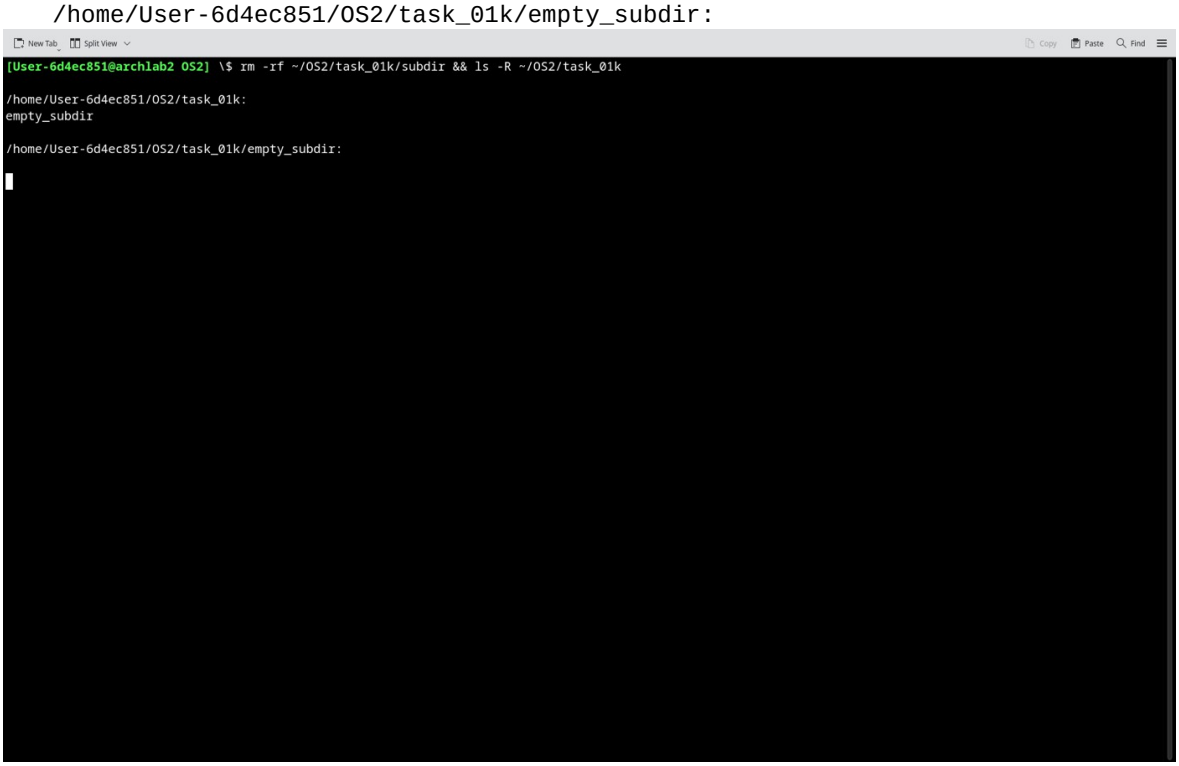


Рисунок 31 - Л6-7 п.1.5, аналог команды rd

5.2 Работа с консольным вводом и выводом

5.2.6 Копирование текста, введённого с консоли, в файл console.txt

В соответствии с заданием Л6-7 п.2.1 выполняется команда:

```
cat > ~/OS2/console.txt
```

Соответствующая команда Windows: copy con

Вывод команды:

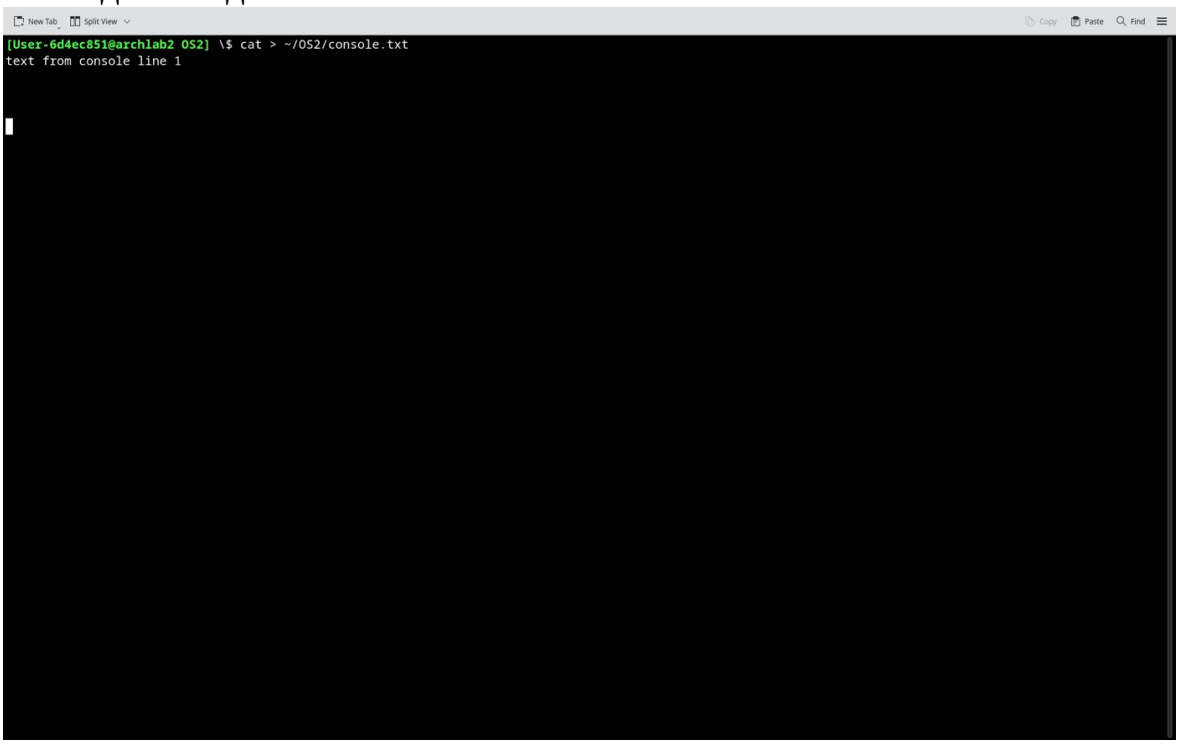


Рисунок 32 - Л6-7 п.2.1, аналог команды copy con

5.2.7 Добавление в конец console.txt данных, вводимых с консоли

В соответствии с заданием Л6-7 п.2.2 выполняется команда:

```
cat >> ~/OS2/console.txt
```

Соответствующая команда Windows: copy con

Вывод команды:

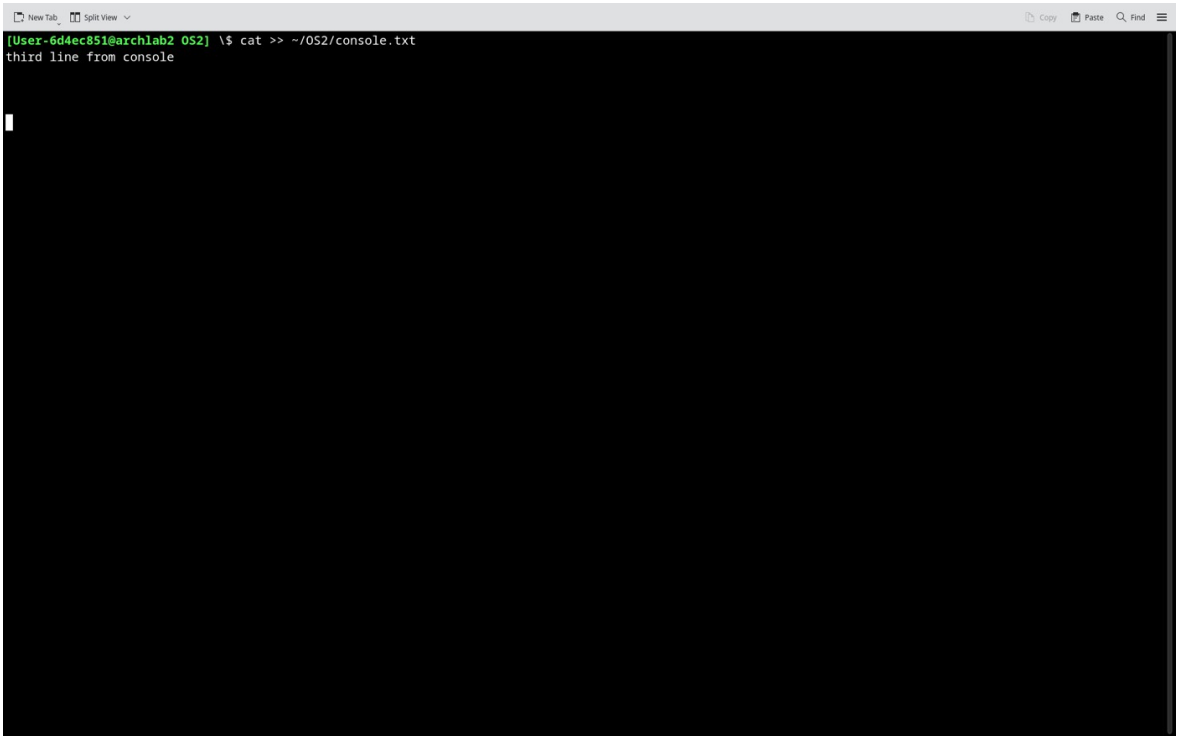


Рисунок 33 - Л6-7 п.2.2, аналог команды copy con

5.3 Сценарии с параметрами

5.3.8 Сценарий bat_01: имя пользователя, дата, время, имя узла

В соответствии с заданием Л6-7 п.3 (bat_01) выполняется команда:

```
cd ~/OS2 && bash bat_01.sh
```

Соответствующая команда Windows: bat_01

Вывод команды:

```
=== bat_01: системные переменные ===
Пользователь: User-6d4ec851
Дата: 2026-10-04
Время: 22:12:03
Компьютер: alarm
```

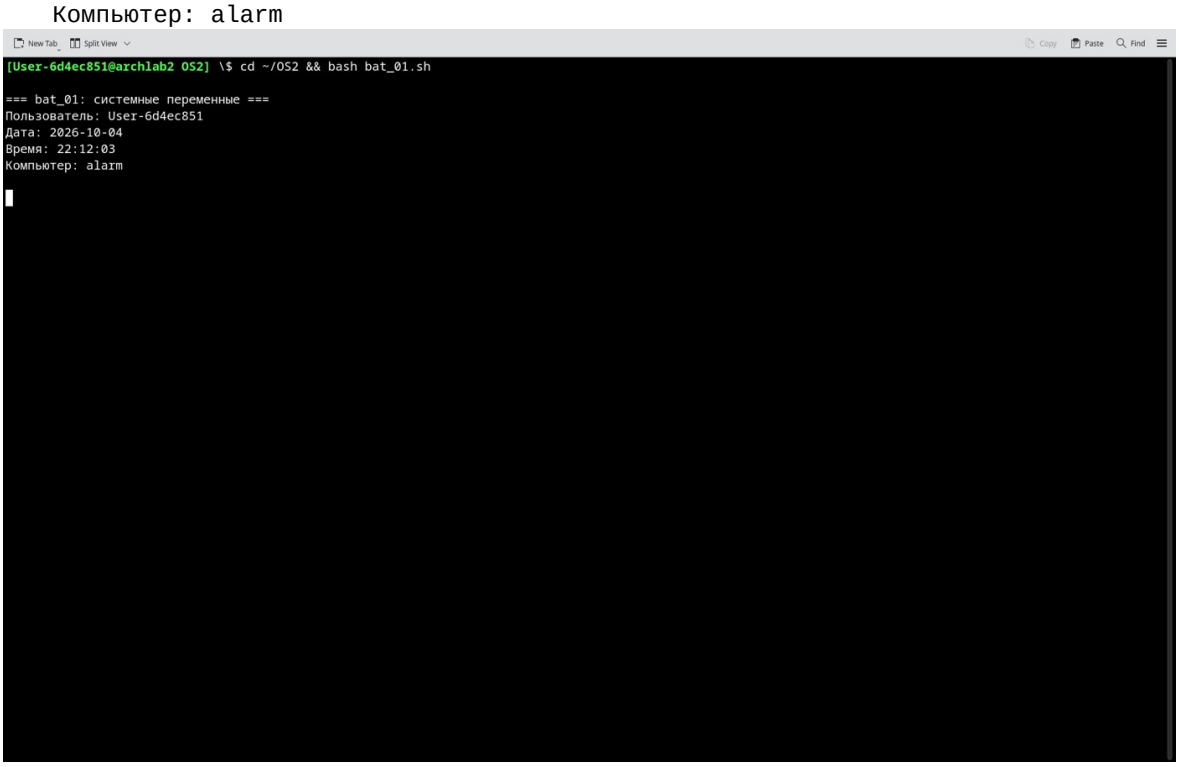


Рисунок 34 - Л6-7 п.3 (bat_01), аналог команды bat_01

5.3.9 Сценарий bat_02: имя файла, время изменения, полный путь

В соответствии с заданием Л6-7 п.3 (bat_02) выполняется команда:

```
cd ~/OS2 && bash bat_02.sh
```

Соответствующая команда Windows: bat_02

Вывод команды:

```
=== bat_02: параметры скрипта ===
Имя файла: bat_02.sh
Изменён: 2026-10-04 21:46:54
Полный путь: /home/User-6d4ec851/OS2/bat_02.sh
```

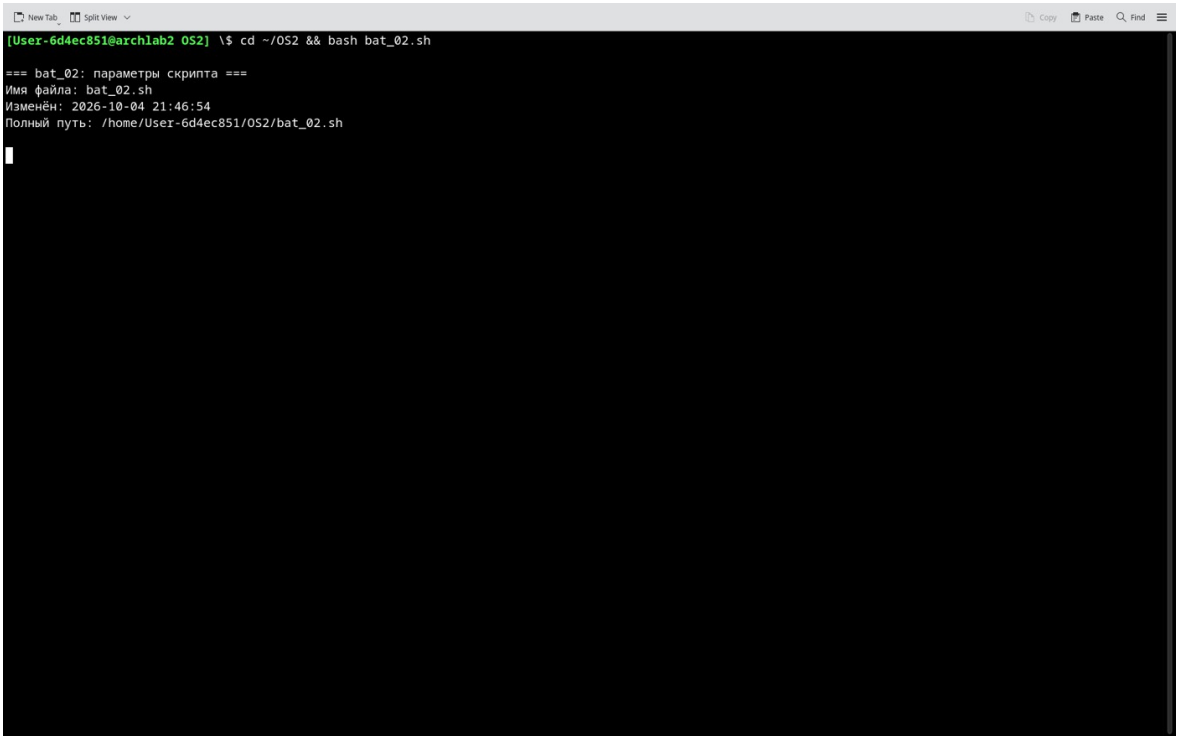


Рисунок 35 - Л6-7 п.3 (bat_02), аналог команды bat_02

5.3.10 Сценарий bat_03: все параметры и первые четыре по отдельности

В соответствии с заданием Л6-7 п.3 (bat_03) выполняется команда:

```
cd ~/OS2 && bash bat_03.sh alpha beta gamma delta epsilon
```

Соответствующая команда Windows: bat_03

Вывод команды:

```
=== bat_03: параметры ===
Все параметры: alpha beta gamma delta epsilon
Первый: alpha
Второй: beta
Третий: gamma
Четвёртый: delta
```

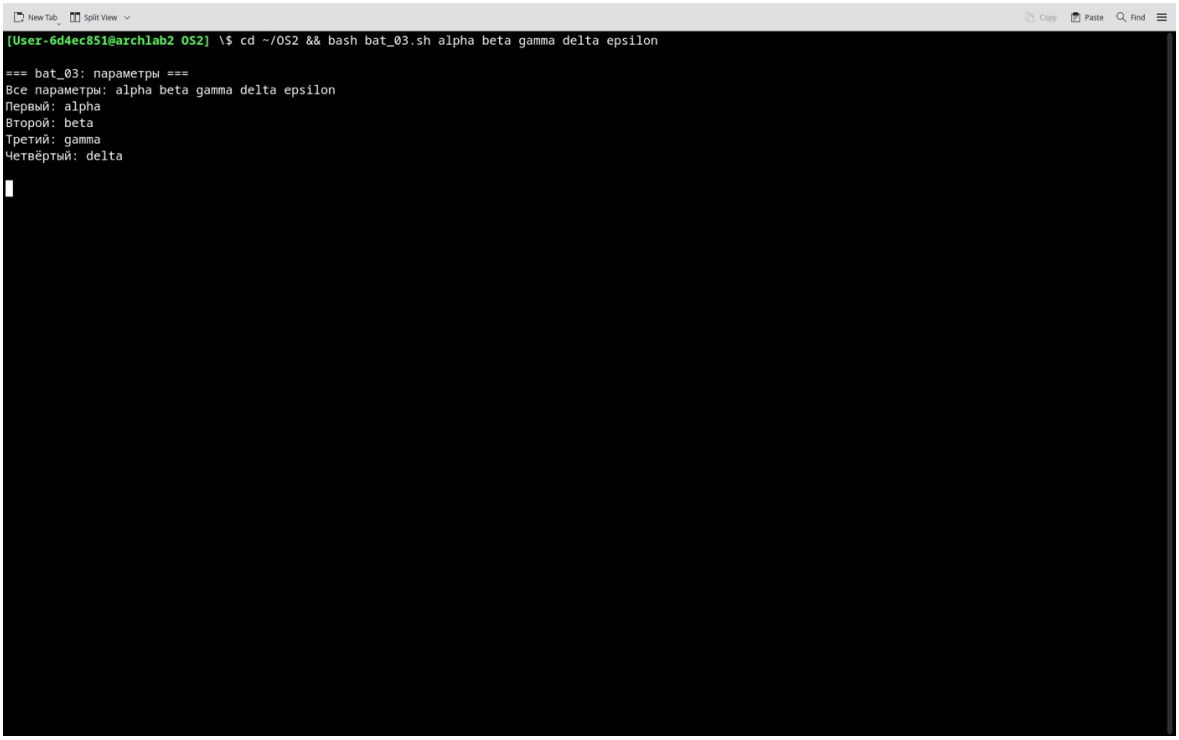


Рисунок 36 - Л6-7 п.3 (bat_03), аналог команды bat_03

5.3.11 Сценарий bat_04: арифметические вычисления с параметрами

В соответствии с заданием Л6-7 п.3 (bat_04) выполняется команда:

```
cd ~/OS2 && bash bat_04.sh 10 4 25
```

Соответствующая команда Windows: bat_04

Вывод команды:

```
=== bat_04: арифметика ===
a=10  b=4  c=25
a-b = 6
a+b = 14
c/b = 6
c*b = 100
(a-b)*(b-a) = -36
```

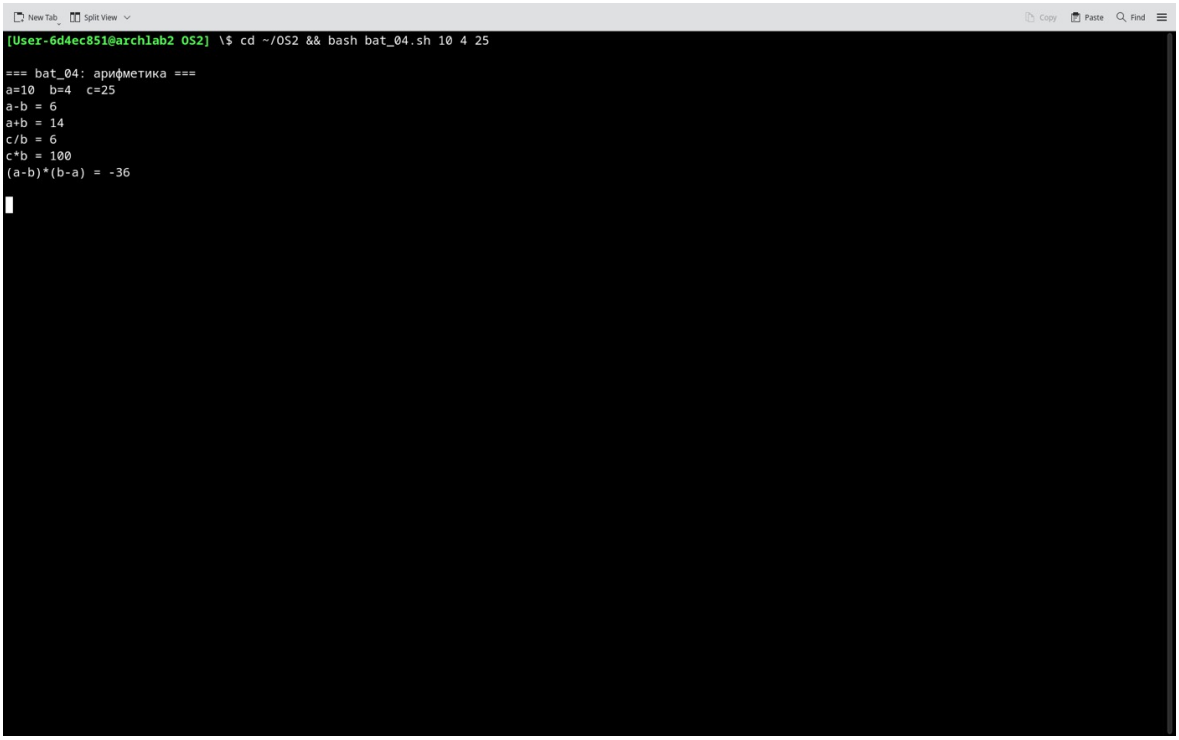


Рисунок 37 - Л6-7 п.3 (bat_04), аналог команды bat_04

5.3.12 Сценарий bat_04: обработка деления на ноль

В соответствии с заданием Л6-7 п.3 (bat_04) выполняется команда:

```
cd ~/OS2 && bash bat_04.sh 10 0 25
```

Соответствующая команда Windows: bat_04

Вывод команды:

```
=== bat_04: арифметика ===
a=10  b=0  c=25
a-b = 10
a+b = 10
c/b — ОШИБКА: деление на ноль невозможно
```

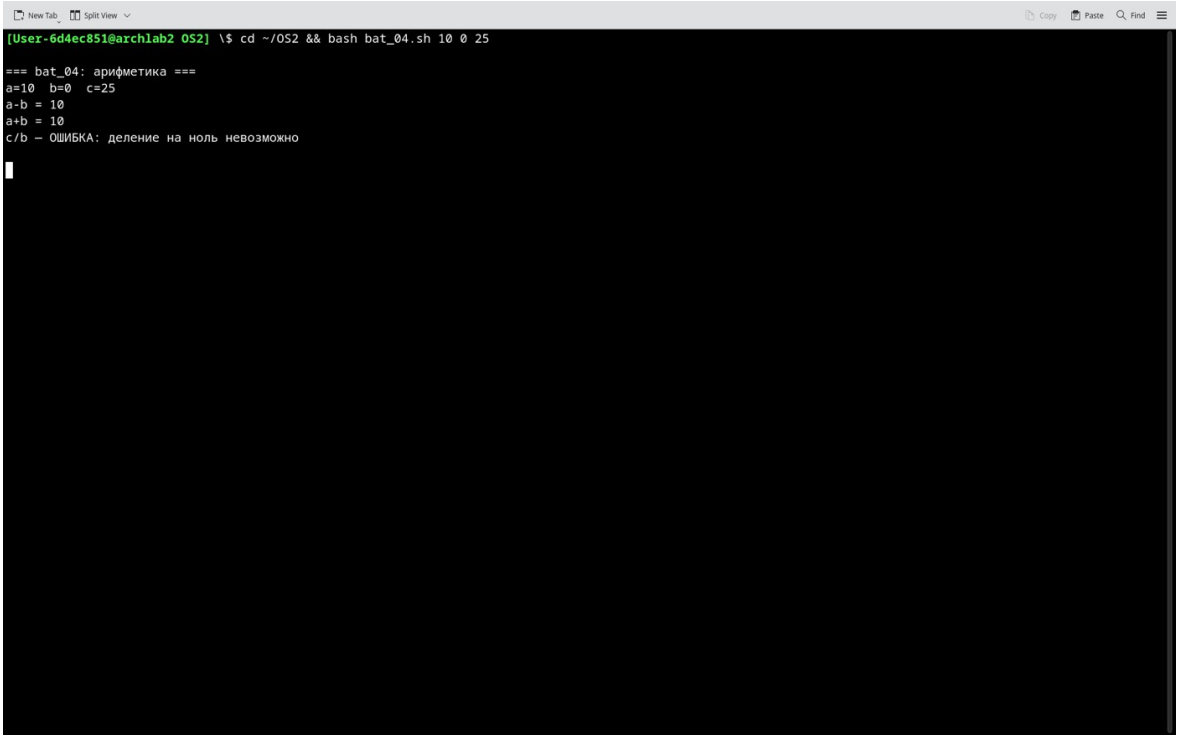


Рисунок 38 - Л6-7 п.3 (bat_04), аналог команды bat_04

5.3.13 Сценарий bat_05: режим создания файла

В соответствии с заданием Л6-7 п.3 (bat_05) выполняется команда:

```
cd ~/OS2 && bash bat_05.sh create newfile.txt && ls -l ~/OS2/newfile.txt
```

Соответствующая команда Windows: bat_05

Вывод команды:

```
Создан файл newfile.txt
-rw-r--r--     1     User-6d4ec851     User-6d4ec851     0     Oct          4     22:12
/home/User-6d4ec851/OS2/newfile.txt
```

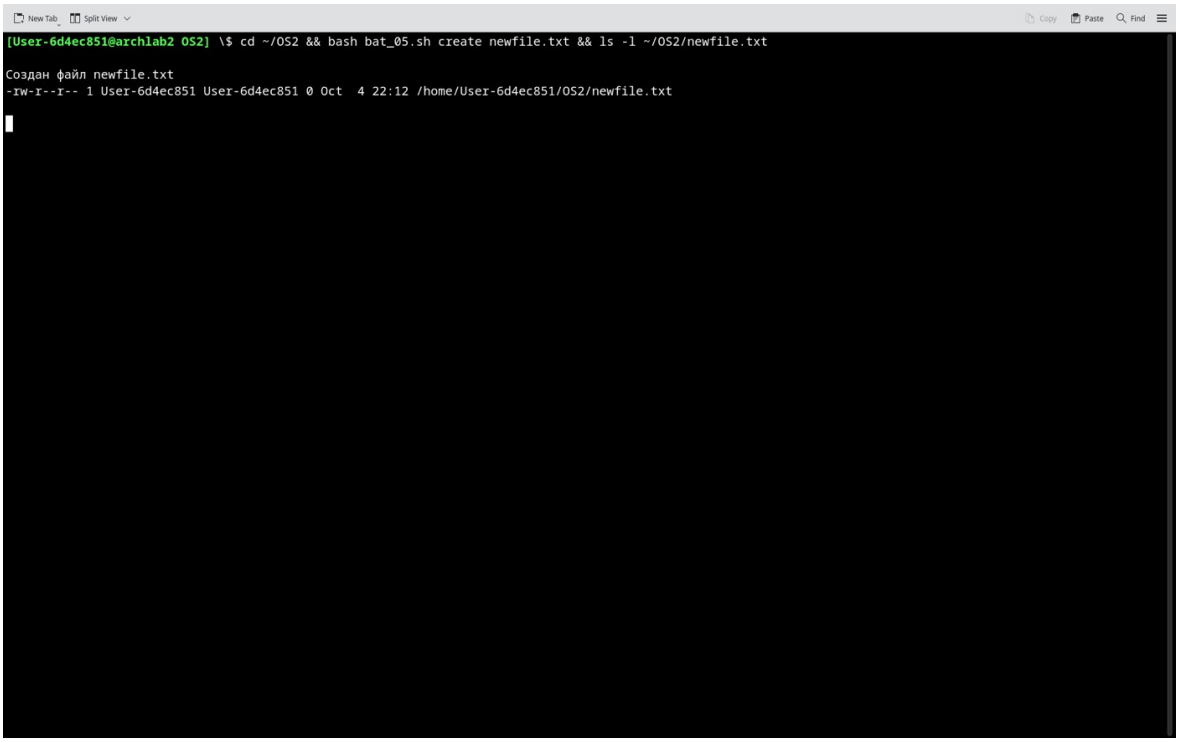


Рисунок 39 - Л6-7 п.3 (bat_05), аналог команды bat_05

5.3.14 Сценарий bat_05: режим удаления файла

В соответствии с заданием Л6-7 п.3 (bat_05) выполняется команда:

```
cd ~/OS2 && bash bat_05.sh delete newfile.txt && ls ~/OS2/newfile.txt
```

Соответствующая команда Windows: bat_05

Вывод команды:

```
Удалён файл newfile.txt
ls: cannot access '/home/User-6d4ec851/OS2/newfile.txt': No such file or directory
```

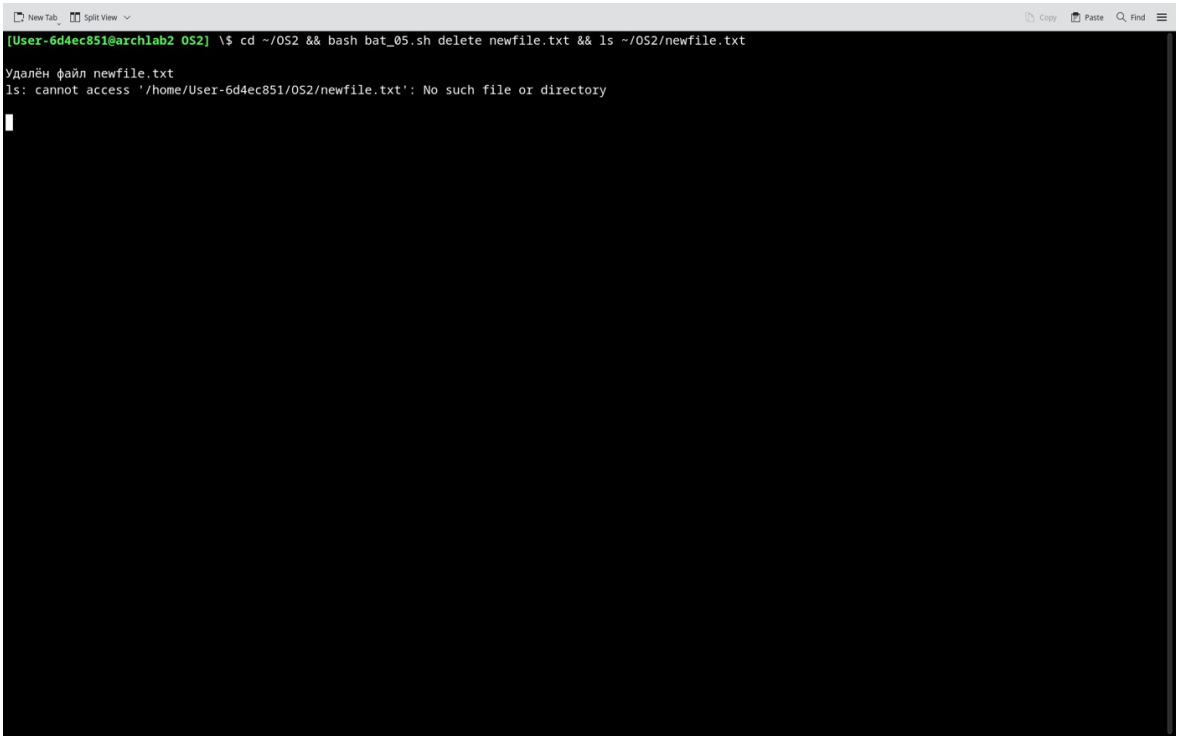


Рисунок 40 - Л6-7 п.3 (bat_05), аналог команды bat_05

5.3.15 Сценарий bat_05: обработка отсутствия параметров

В соответствии с заданием Л6-7 п.3 (bat_05) выполняется команда:

```
cd ~/OS2 && bash bat_05.sh
```

Соответствующая команда Windows: bat_05

Вывод команды:

```
Использование: bat_05.sh create|delete <файл>
```

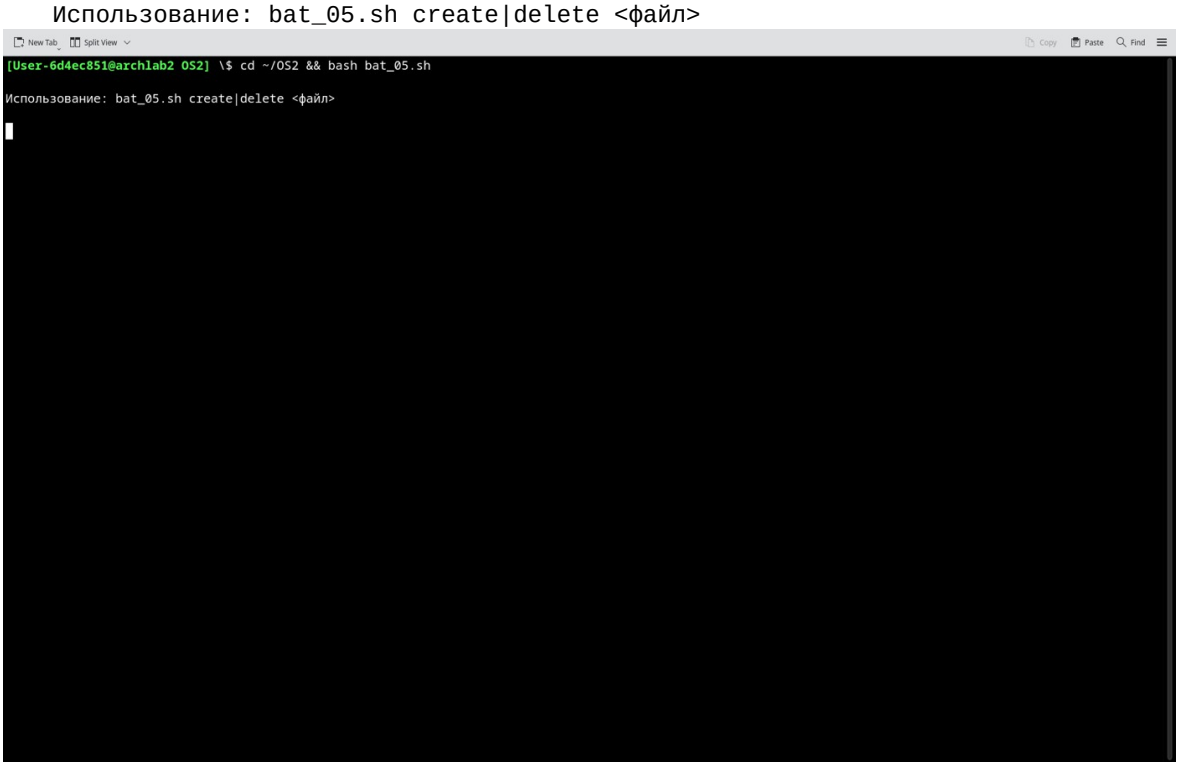


Рисунок 41 - Л6-7 п.3 (bat_05), аналог команды bat_05

5.3.16 Сценарий bat_05: обработка неизвестного режима

В соответствии с заданием Л6-7 п.3 (bat_05) выполняется команда:

```
cd ~/OS2 && bash bat_05.sh unknown newfile.txt
```

Соответствующая команда Windows: bat_05

Вывод команды:

```
Неизвестный режим: unknown (допустимо: create, delete)
```

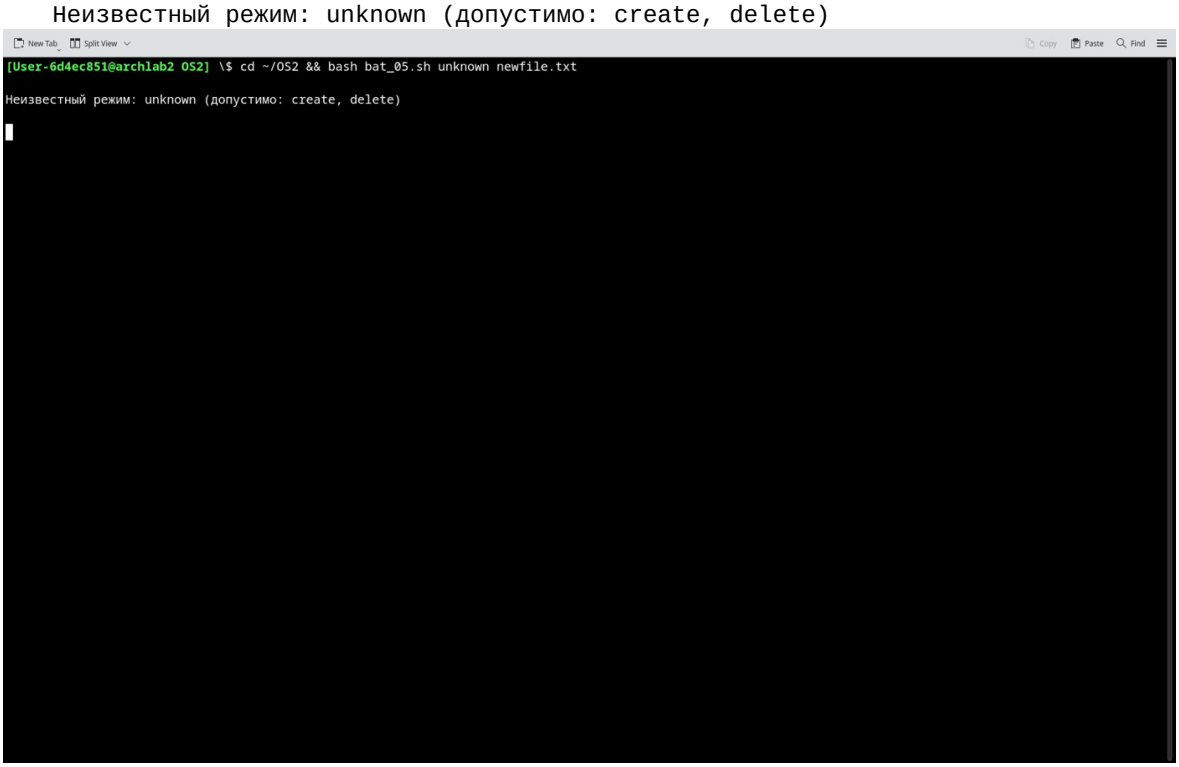


Рисунок 42 - Л6-7 п.3 (bat_05), аналог команды bat_05

5.4 Исследование переменных окружения

5.4.17 Вывод всех переменных окружения

В соответствии с заданием Л6-7 п.4.3 выполняется команда:

```
env | sort
```

Соответствующая команда Windows: set

Вывод команды:

```
COLORFGBG=15;0
COLORTERM=truecolor
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1001/bus
DISPLAY=:1
DOTNET_ROOT=/home/User-6d4ec851/.dotnet
FLATPAK_TTY_PROGRESS=1
HOME=/home/User-6d4ec851
KONSOLE_DBUS_ACTIVATION_COOKIE=7CFZ/yMcfDZ1KEa4QOg2ji5LbagqG70NxdcK2E+G3R4=
KONSOLE_DBUS_SERVICE=:1.1025
KONSOLE_DBUS_SESSION=/Sessions/1
KONSOLE_DBUS_WINDOW=/Windows/1
KONSOLE_VERSION=260801
LANG=C
LANGUAGE=
LOGNAME=User-6d4ec851
MAIL=/var/spool/mail/User-6d4ec851
MOTD_SHOWN=pam
PATH=/home/User-6d4ec851/.dotnet:/home/User-6d4ec851/.dotnet:/usr/local/sbin:/usr/
local/bin:/usr/bin:/usr/bin/site_perl:/usr/bin/vendor_perl:/usr/bin/core_perl
PROFILEHOME=
PWD=/home/User-6d4ec851
SHELL=/bin/bash
SHELL_SESSION_ID=6b0ecb27012749acb89036ffe35a1181
SHLVL=1
SSH_CLIENT=192.168.78.1 55338 22
SSH_CONNECTION=192.168.78.1 55338 192.168.78.154 22
TERM=xterm-256color
... ещё 9 строк вывода
```

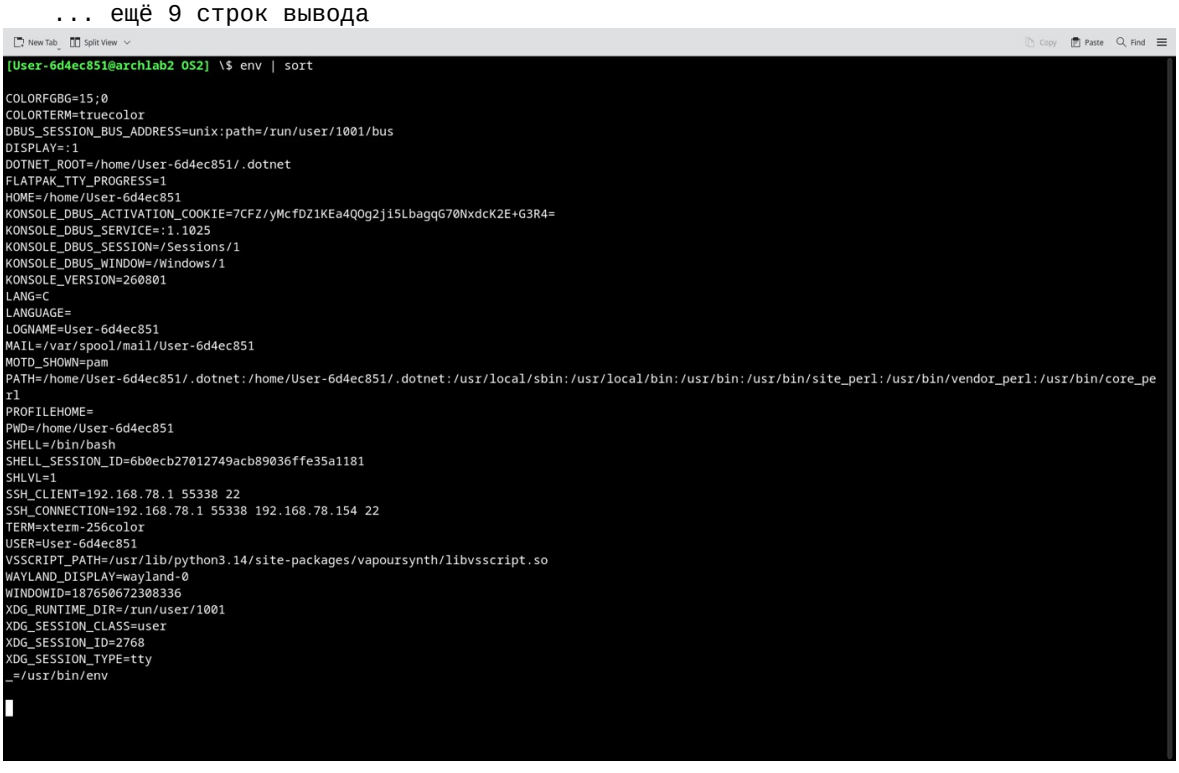


Рисунок 43 - Л6-7 п.4.3, аналог команды set

5.4.18 Анализ ключевых переменных окружения

В соответствии с заданием Л6-7 п.4.3 выполняется команда:

```
env | sort | grep -E «^(PATH|HOME|USER|HOSTNAME|SHELL|TMPDIR|LANG|TERM)=»
```

Соответствующая команда Windows: echo %PATH%

Вывод команды:

```
HOME=/home/User-6d4ec851
LANG=C
PATH=/home/User-6d4ec851/.dotnet:/home/User-6d4ec851/.dotnet:/usr/local/sbin:/usr/
local/bin:/usr/bin:/usr/bin/site_perl:/usr/bin/vendor_perl:/usr/bin/core_perl
SHELL=/bin/bash
TERM=xterm-256color
USER=User-6d4ec851
```

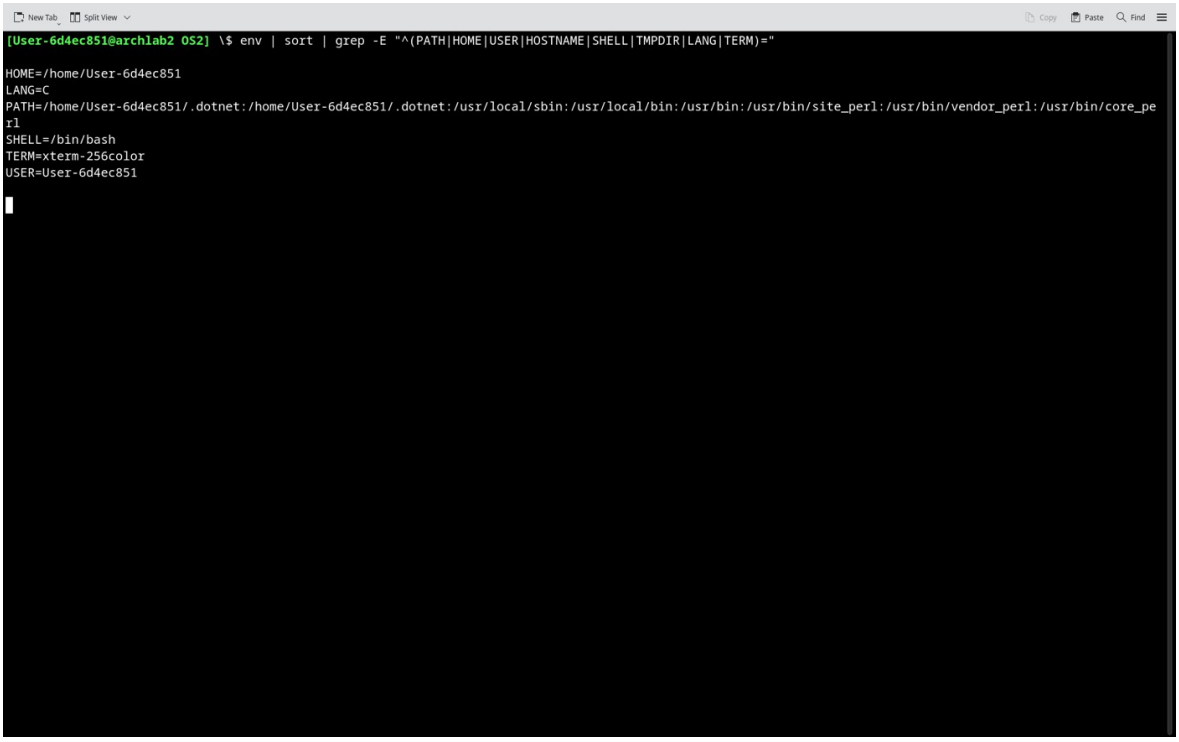


Рисунок 44 - Л6-7 п.4.3, аналог команды echo %PATH%

5.4.19 Определение значения PATH и подсчёт каталогов

В соответствии с заданием Л6-7 п.4.3 выполняется команда:

```
printenv PATH; echo «---»; echo $PATH | tr ':' '\n' | wc -l
```

Соответствующая команда Windows: echo %PATH%

Вывод команды:

```
/home/User-6d4ec851/.dotnet:/home/User-6d4ec851/.dotnet:/usr/local/sbin:/usr/
local/bin:/usr/bin:/usr/bin/site_perl:/usr/bin/vendor_perl:/usr/bin/core_perl
---
8
```

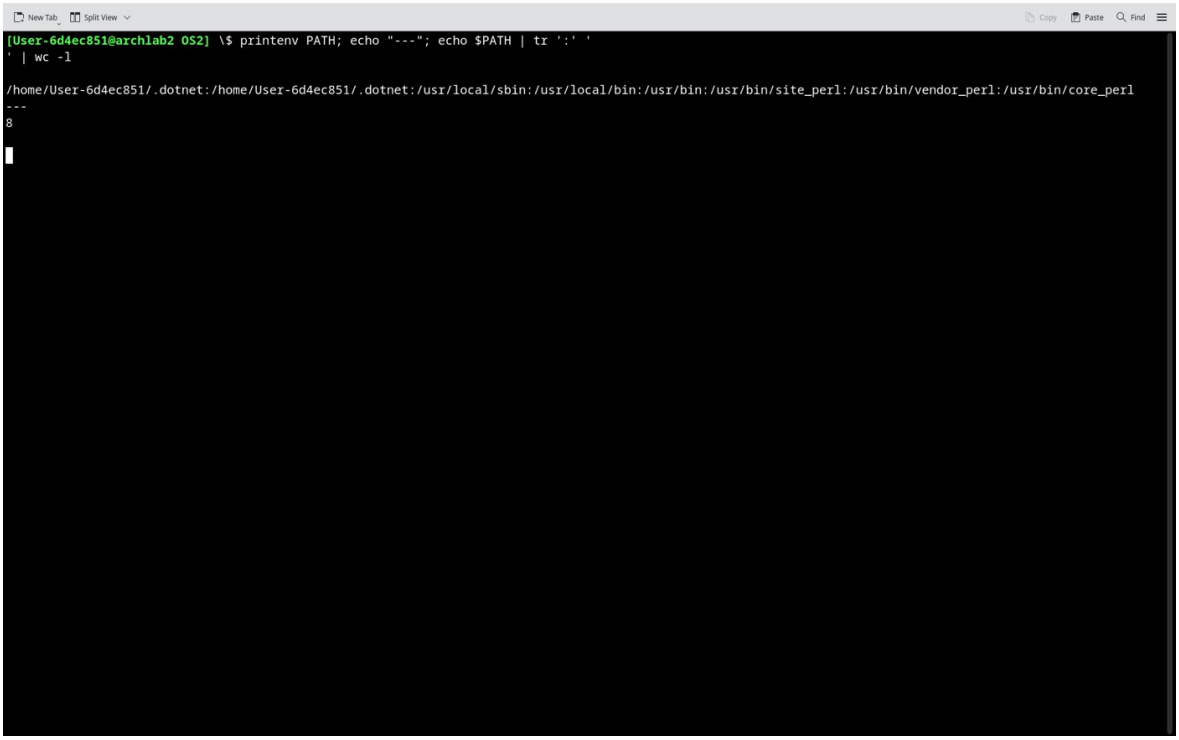


Рисунок 45 - Л6-7 п.4.3, аналог команды echo %PATH%

Вывод по разделу 5

В разделе 5 выполнено 19 заданий. Каждое задание сопоставлено с командой Windows-части, зафиксировано снимком экрана и сохранено текстовым выводом. Выполнение подтверждено автоматизированной проверкой распознавания текста на снимках: во всех кадрах присутствуют строка с введённой командой и результат её выполнения.

Задания раздела 5 выполнены полностью, команды отработали без ошибок, кроме заданий, где ошибочная ситуация является частью проверяемого сценария.

6 ВЫПОЛНЕНИЕ ЗАДАНИЙ УРОВНЕЙ 8-9

Задания уровней 8-9 выполнены в консоли Arch Linux ARM. Реализованы скрипты подсчёта команд в переменной PATH, вывода дерева каталогов с размерами файлов в десятичных и двоичных единицах, а также монитор процессов топ-5 по загрузке процессора и памяти с поддержкой параметра интервала обновления.

6.1 Подсчёт команд в PATH и дерево каталогов

6.1.1 Подсчёт исполняемых и неисполняемых команд в PATH

В соответствии с заданием Л8 п.2.1 (Linux) выполняется команда:

```
cd ~/OS2 && ./path_count.sh | head -30
```

Соответствующая команда Windows: —

Вывод команды:

```
============================================================
 СТАТИСТИКА ПО КАТАЛОГАМ PATH
============================================================
КАТАЛОГ                                              EXEC NON-EXEC
------------------------------------------------------------ -------- --------
/home/User-6d4ec851/.dotnet                                     1       10
/home/User-6d4ec851/.dotnet                                     1       10
/usr/local/sbin                                                 0        0
/usr/local/bin                                                  0        0
/usr/bin                                                     1891        0
/usr/bin/site_perl                                              0        0
/usr/bin/vendor_perl                                            0        0
/usr/bin/core_perl                                             29        0
============================================================
Каталогов в PATH:               8 (из них не существует: 0)
Всего исполняемых файлов:       1922
Всего неисполняемых файлов:     20
Всего файлов в PATH:            1942
============================================================
```

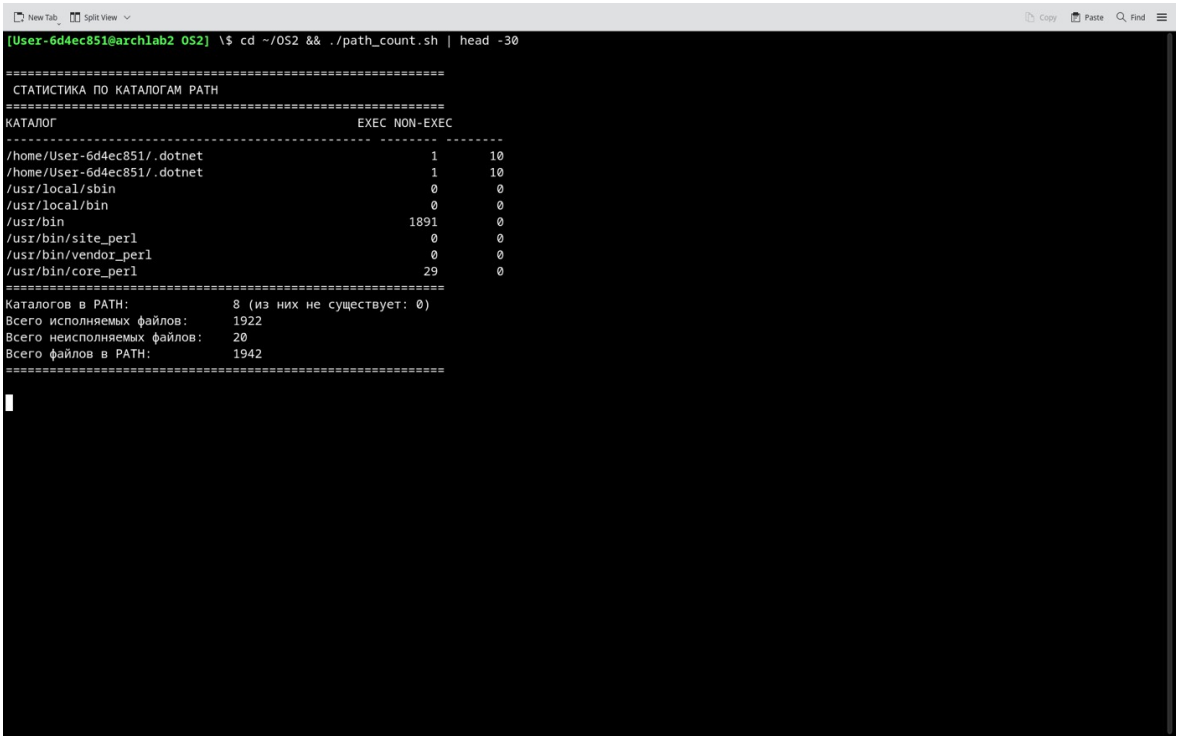


Рисунок 46 - Л8 п.2.1 (Linux), аналог команды —

6.1.2 Вывод дерева каталогов с размерами файлов

В соответствии с заданием Л8 п.2.2 (Linux) выполняется команда:

```
cd ~/OS2 && ./tree_sizes.sh task_01b
```

Соответствующая команда Windows: —

Вывод команды:

```
    ================================================================
     ДЕРЕВО КАТАЛОГОВ: task_01b
     Формат размера файла: <десятичный> / <двоичный>
    ================================================================
    [DIR ] .                                          файлов: 0, размер: 0 B / 0 B
    [DIR ] task_01b                                   файлов: 9, размер: 56 B / 56 B
      [FILE] archive-readonly.txt                       8 B / 8 B
      [DIR ] hidden_dir                                 файлов: 4, размер: 23 B / 23 B
        [DIR ] empty_subdir                               файлов: 0, размер: 0 B / 0 B
        [DIR ] subdir                                     файлов: 2, размер: 10 B / 10 B
          [FILE] txt1.txt                                   5 B / 5 B
          [FILE] txt2.txt                                   5 B / 5 B
        [FILE] system1.txt                                8 B / 8 B
        [FILE] txt1.txt                                   5 B / 5 B
      [FILE] plain.txt                                  6 B / 6 B
      [FILE] readonly.txt                               9 B / 9 B
      [DIR ] subdir                                     файлов: 2, размер: 10 B / 10 B
        [DIR ] empty_subdir                               файлов: 0, размер: 0 B / 0 B
        [FILE] txt1.txt                                   5 B / 5 B
        [FILE] txt2.txt                                   5 B / 5 B
    ================================================================
```

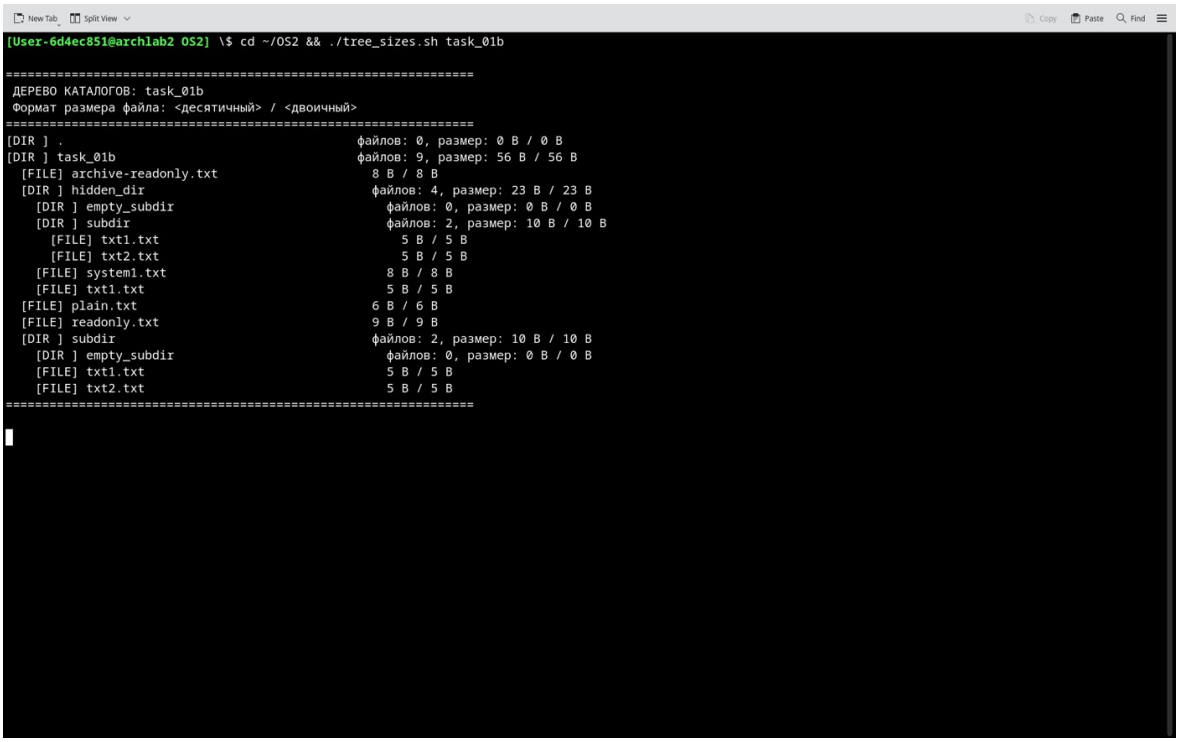


Рисунок 47 - Л8 п.2.2 (Linux), аналог команды —

6.1.3 Вывод дерева всего рабочего каталога

В соответствии с заданием Л8 п.2.2 (Linux) выполняется команда:

```
cd ~/OS2 && ./tree_sizes.sh . | head -34
```

Соответствующая команда Windows: —

Вывод команды:

```
    ================================================================
     ДЕРЕВО КАТАЛОГОВ: .
     Формат размера файла: <десятичный> / <двоичный>
    ================================================================
    [DIR ] .                                          файлов: 0, размер: 0 B / 0 B
    [DIR ] .                                               файлов: 60, размер: 20.47 KB /
19.99 KiB
      [FILE] User-6d4ec851.bmp                          5 B / 5 B
      [FILE] bat_01.sh                                  354 B / 354 B
      [FILE] bat_02.sh                                  451 B / 451 B
      [FILE] bat_03.sh                                  390 B / 390 B
```

```
      [FILE] bat_04.sh                                658 B / 658 B
      [FILE] bat_05.sh                                878 B / 878 B
      [FILE] calc.sh                                  3.43 KB / 3.35 KiB
      [FILE] path_count.sh                            1.53 KB / 1.49 KiB
      [FILE] prep.sh                                  5.58 KB / 5.45 KiB
      [DIR ] task_01a                                 файлов: 0, размер: 0 B / 0 B
      [DIR ] task_01b                                 файлов: 9, размер: 56 B / 56 B
        [FILE] archive-readonly.txt                     8 B / 8 B
        [DIR ] hidden_dir                               файлов: 4, размер: 23 B / 23 B
          [DIR ] empty_subdir                             файлов: 0, размер: 0 B / 0 B
          [DIR ] subdir                                   файлов: 2, размер: 10 B / 10
B
            [FILE] txt1.txt                                 5 B / 5 B
            [FILE] txt2.txt                                 5 B / 5 B
          [FILE] system1.txt                              8 B / 8 B
          [FILE] txt1.txt                                 5 B / 5 B
        [FILE] plain.txt                                6 B / 6 B
    ... ещё 8 строк вывода
```

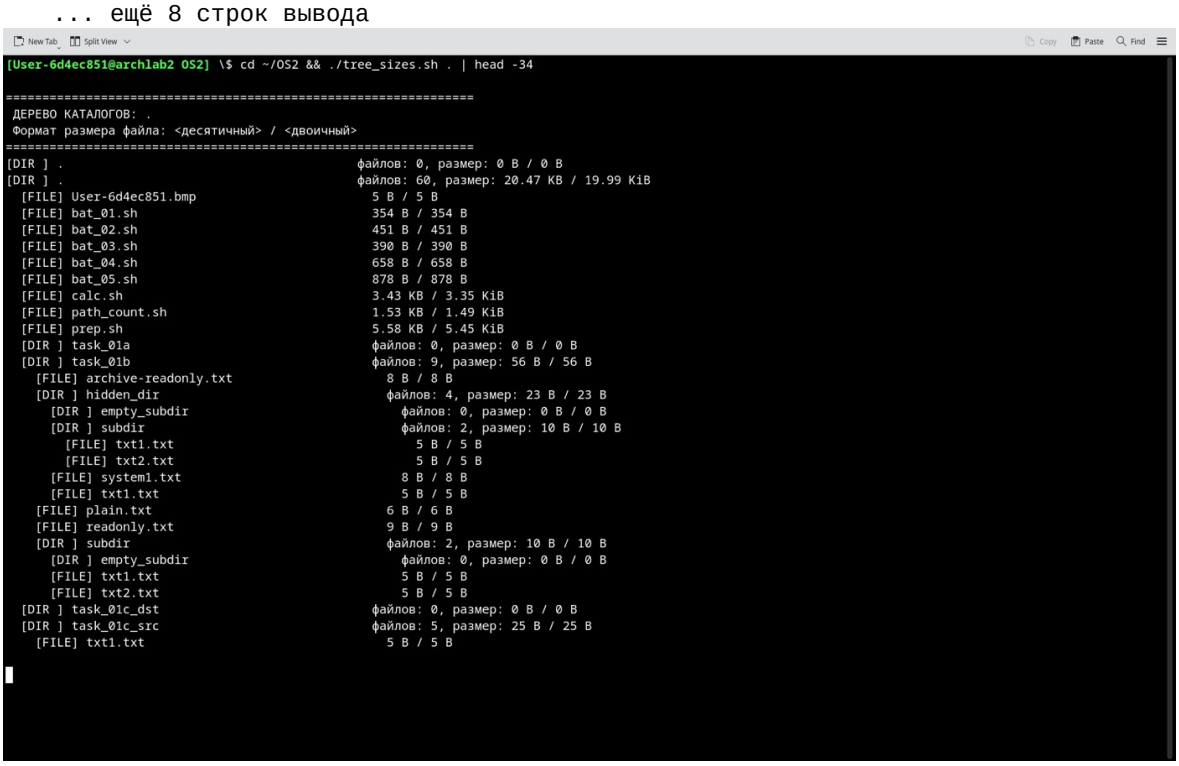


Рисунок 48 - Л8 п.2.2 (Linux), аналог команды —

6.2 Монитор процессов топ-5

6.2.4 Монитор топ-5 процессов с интервалом обновления по умолчанию

В соответствии с заданием Л9 (интервал 5 с) выполняется команда:

```
cd ~/OS2 && timeout 8 ./top5.sh | head -30
```

Соответствующая команда Windows: —

Вывод команды:

```
================================================================
 МОНИТОР ПРОЦЕССОВ - ТОП-5
 Время:        2026-10-04 22:13:00
 Интервал:     5 с (Ctrl+C - выход)
 Всего памяти: 7415 МБ
================================================================
 ТОП-5 ПРОЦЕССОВ ПО ЗАГРУЗКЕ ЦП
PID      ИМЯ                       CPU,% ПАМЯТЬ,МБ ВРЕМЯ CPU
-------- ------------------------ --------  --------- ----------
125537   baloo_file_extr              33.3       29.7 00:00:00
785      kwin_wayland                  0.5      229.9 00:01:02
100877   kworker/u16:6-e               0.2        0.0 00:00:03
71819    kworker/u16:1-e               0.2        0.0 00:00:08
893      plasmashell                   0.1      341.1 00:00:21

 ТОП-5 ПРОЦЕССОВ ПО ЗАГРУЗКЕ ОПЕРАТИВНОЙ ПАМЯТИ
PID      ИМЯ                       CPU,% ПАМЯТЬ,МБ ВРЕМЯ CPU
-------- ------------------------ --------  --------- ----------
893      plasmashell                   0.1      341.1 00:00:21
785      kwin_wayland                  0.5      229.9 00:01:02
79997    krunner                       0.0      200.2 00:00:01
119535   konsole                       0.1       96.4 00:00:00
453      Xorg                          0.0       81.6 00:00:00

================================================================
================================================================
... ещё 3 строк вывода
```

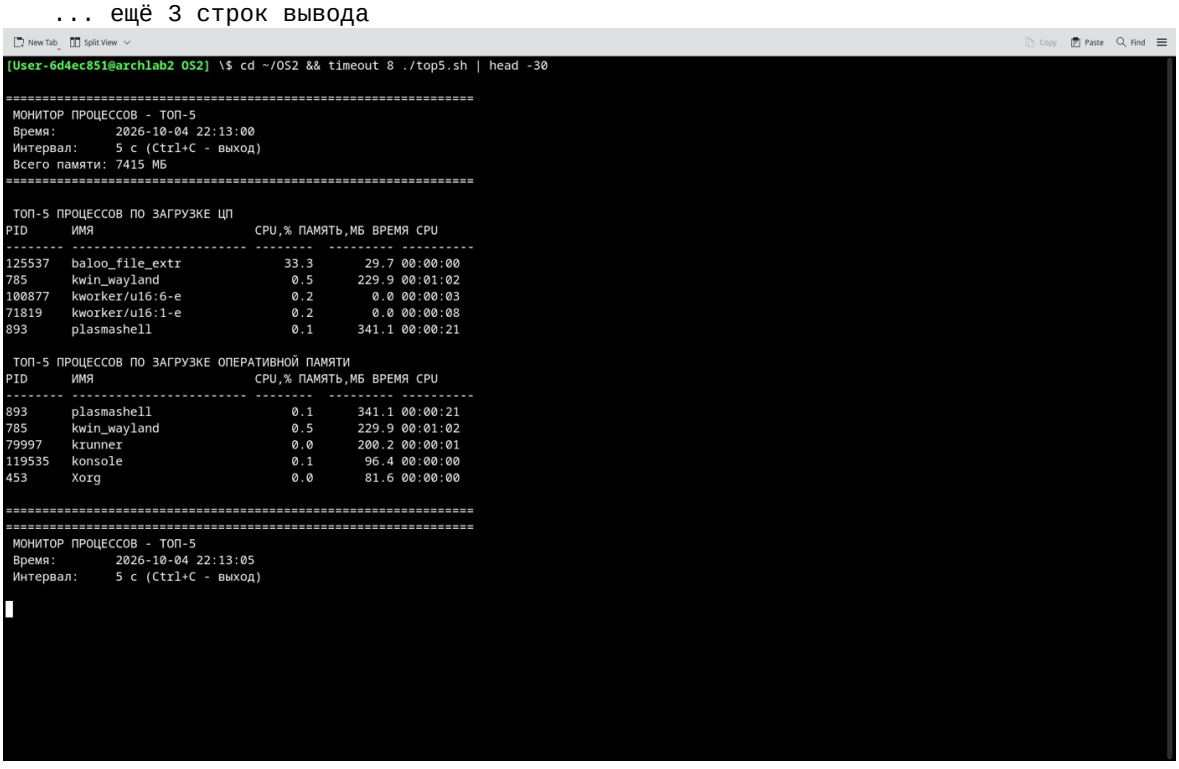


Рисунок 49 - Л9 (интервал 5 с), аналог команды —

6.2.5 Монитор топ-5 процессов с интервалом обновления 3 секунды

В соответствии с заданием Л9 (-Interval 3) выполняется команда:

```
cd ~/OS2 && timeout 8 ./top5.sh 3 | head -30
```

Соответствующая команда Windows: —

Вывод команды:

```
================================================================
 МОНИТОР ПРОЦЕССОВ - ТОП-5
 Время:        2026-10-04 22:13:09
 Интервал:     3 с (Ctrl+C - выход)
 Всего памяти: 7415 МБ
================================================================
 ТОП-5 ПРОЦЕССОВ ПО ЗАГРУЗКЕ ЦП
PID      ИМЯ                       CPU,% ПАМЯТЬ,МБ ВРЕМЯ CPU
-------- ------------------------ --------  --------- ----------
125717   baloo_file_extr               3.2       29.7 00:00:00
785      kwin_wayland                  0.5      229.9 00:01:02
100877   kworker/u16:6-w               0.2        0.0 00:00:03
71819    kworker/u16:1-w               0.2        0.0 00:00:08
893      plasmashell                   0.1      341.1 00:00:21

 ТОП-5 ПРОЦЕССОВ ПО ЗАГРУЗКЕ ОПЕРАТИВНОЙ ПАМЯТИ
PID      ИМЯ                       CPU,% ПАМЯТЬ,МБ ВРЕМЯ CPU
-------- ------------------------ --------  --------- ----------
893      plasmashell                   0.1      341.1 00:00:21
785      kwin_wayland                  0.5      229.9 00:01:02
79997    krunner                       0.0      200.2 00:00:01
119535   konsole                       0.1       96.4 00:00:00
453      Xorg                          0.0       81.6 00:00:00

================================================================
================================================================
... ещё 3 строк вывода
```

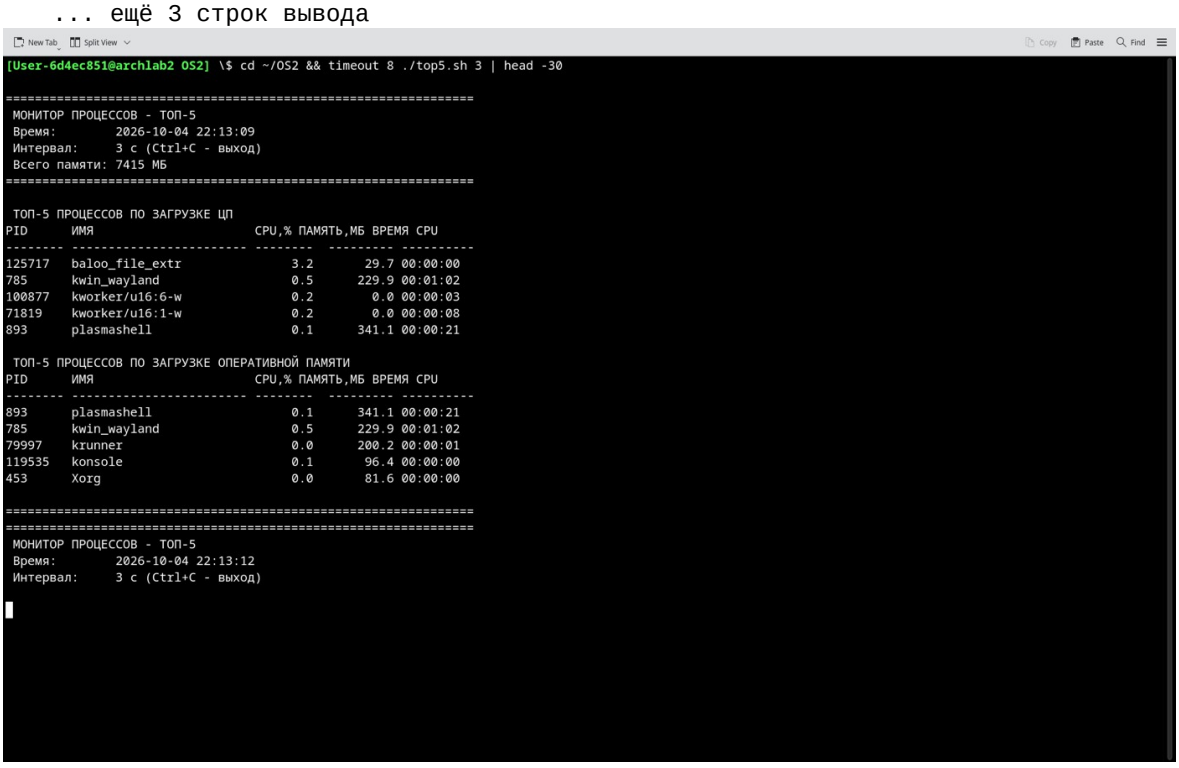


Рисунок 50 - Л9 (-Interval 3), аналог команды —

6.2.6 Монитор топ-5 процессов: обработка ошибочного параметра

В соответствии с заданием Л9 (ошибка ввода) выполняется команда:

```
cd ~/OS2 && ./top5.sh abc
```

Соответствующая команда Windows: —

Вывод команды:

```
Ошибка: интервал должен быть положительным целым числом секунд
```

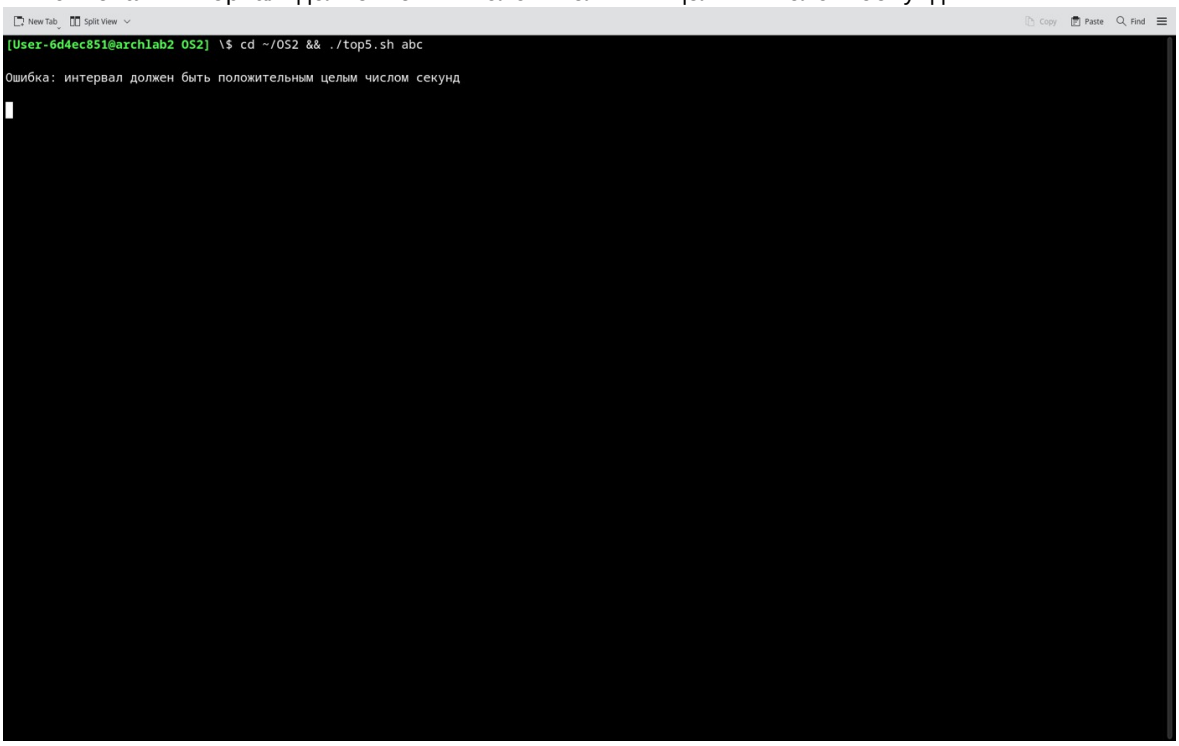


Рисунок 51 - Л9 (ошибка ввода), аналог команды —

Вывод по разделу 6

В разделе 6 выполнено 6 заданий. Каждое задание сопоставлено с командой Windows-части, зафиксировано снимком экрана и сохранено текстовым выводом. Выполнение подтверждено автоматизированной проверкой распознавания текста на снимках: во всех кадрах присутствуют строка с введённой командой и результат её выполнения.

Задания раздела 6 выполнены полностью, команды отработали без ошибок, кроме заданий, где ошибочная ситуация является частью проверяемого сценария.

ЗАКЛЮЧЕНИЕ

В ходе работы выполнено 51 заданий лабораторной работы на уровнях 4-9 в командной оболочке bash операционной системы Arch Linux ARM. Все задания Windows-части выполнены средствами Linux, что позволило представить единый отчёт по лабораторной работе.

Освоены приёмы навигации по файловой системе с использованием абсолютных и относительных путей, создания и удаления каталогов, копирования, перемещения и переименования файлов, сравнения файлов и удаления по признакам. Изучены конвейеры и перенаправление вывода, способы интерактивного ввода данных с клавиатуры и сохранения их в файл.

Составлены и проверены сценарии с параметрами: получение системных сведений, обработка аргументов, арифметические вычисления с обработкой деления на ноль, работа в режимах создания и удаления файлов. Исследованы переменные окружения, определена роль переменной PATH и выполнен подсчёт команд в ней.

Разработаны скрипты подсчёта исполняемых и неисполняемых команд в каталогах PATH, вывода дерева каталогов с размерами файлов в десятичных и двоичных единицах измерения, а также монитор топ-5 процессов по загрузке процессора и памяти с поддержкой параметра интервала обновления.

Цель работы достигнута: приёмы работы с командной оболочкой освоены, сценарии автоматизации составлены и проверены на реальной машине. Дальнейшее развитие работы может включать разработку собственной встроенной команды shell в соответствии с заданием уровня 10.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Методические указания по выполнению лабораторной работы № 2 «Работа в командной оболочке» : БГТУ, 2026. - 12 с.

[2] Arch Linux : документация проекта. - URL: https://wiki.archlinux.org/ (дата обращения: 05.10.2026).

[3] GNU Bash Reference Manual : Free Software Foundation. - URL: https://www.gnu.org/software/bash/manual/ (дата обращения: 05.10.2026).

ПРИЛОЖЕНИЕ А

(Листинги сценариев)

В каталоге lab2/linux размещены файлы, обеспечивающие подготовку среды, выполнение заданий и сборку настоящего отчёта: скрипт prep.sh для подготовки исходной структуры рабочего каталога, драйвер run_lab.sh для выполнения команд и съёмки снимков экрана, оператор push_cmds.py для передачи команд в очередь драйвера, файлы cmds_4-5.tsv, cmds_6-7.tsv и cmds_8-9.tsv со списками выполняемых команд, модуль снимки_разметка.py с привязкой снимков к пунктам задания и настоящий генератор отчёта. Результаты выполнения сохранены в каталогах скрины и выводы.

A.1 Сценарий вывода системных переменных окружения (аналог bat_01)

```
#!/bin/bash
# bat_01.sh — аналог bat_01.bat: имя пользователя, дата, время, имя компьютера


echo «=== bat_01: системные переменные ===»
echo «Пользователь: $USER»
echo «Дата: $(date '+%Y-%m-%d')»
echo «Время: $(date '+%H:%M:%S')»
echo «Компьютер: $(uname -n)»
```

A.2 Сценарий получения сведений о файле сценария (аналог bat_02)

```
#!/bin/bash
# bat_02.sh — аналог bat_02.bat: имя скрипта, время изменения, полный путь
# Аналоги bat: %~nx0 (имя файла), %~t0 (дата изменения), %~f0 (полный путь)


echo «=== bat_02: параметры скрипта ===»
echo «Имя файла: $(basename »$0«)»
echo «Изменён: $(stat -c '%y' »$0« | cut -d'.' -f1)»
echo «Полный путь: $(realpath »$0«)»
```

A.3 Сценарий работы с параметрами командной строки (аналог bat_03)

```
#!/bin/bash
# bat_03.sh — аналог bat_03.bat: все параметры и первые четыре по отдельности
# Аналоги bat: %* (все), %1..%4 (по отдельности)


echo «=== bat_03: параметры ===»
echo «Все параметры: $*»
echo «Первый: $1»
echo «Второй: $2»
echo «Третий: $3»
echo «Четвёртый: $4»
```

A.4 Сценарий арифметических вычислений с обработкой деления на ноль (аналог bat_04)

```
#!/bin/bash
# bat_04.sh — аналог bat_04.bat: a-b, a+b, c/b, c*b, (a-b)*(b-a)
# a, b, c — первый, второй и третий параметры. Проверка деления на ноль.


if [ $# -ne 3 ]; then
    echo «Использование: $0 <a> <b> <c>»
    exit 1
fi


a=$1
b=$2
c=$3


echo «=== bat_04: арифметика ===»
echo «a=$a  b=$b  c=$c»
echo «a-b = $(( a - b ))»
echo «a+b = $(( a + b ))»


if [ «$b» -eq 0 ]; then
    echo «c/b — ОШИБКА: деление на ноль невозможно»
    exit 1
```

```
fi


echo «c/b = $(( c / b ))»
echo «c*b = $(( c * b ))»
echo «(a-b)*(b-a) = $(( (a - b) * (b - a) ))»
```

A.5 Сценарий работы в режимах создания и удаления файла (аналог bat_05)

```
#!/bin/bash
# bat_05.sh — аналог bat_05.bat: режимы create/delete
# Проверка отсутствия параметров, проверка существования файла.


if [ $# -ne 2 ]; then
    echo «Использование: $0 create|delete <файл>»
    exit 1
fi


action=$1
file=$2


case «$action» in
    create)
        if [ -e «$file» ]; then
            echo «Файл $file уже существует»
        else
            touch «$file» && echo «Создан файл $file»
        fi
        ;;
    delete)
        if [ -e «$file» ]; then
            rm «$file» && echo «Удалён файл $file»
        else
            echo «Файл $file не существует»
        fi
        ;;
    *)
        echo «Неизвестный режим: $action (допустимо: create, delete)»
        exit 1
        ;;
esac
```

A.6 Сценарий подсчёта исполняемых и неисполняемых команд в каталогах PATH

```
#!/bin/bash
# path_count.sh — подсчёт исполняемых и неисполняемых команд в PATH (уровень 8)


echo «============================================================»
echo « СТАТИСТИКА ПО КАТАЛОГАМ PATH»
echo «============================================================»
printf «%-50s %8s %8s\n» «КАТАЛОГ» «EXEC» «NON-EXEC»
printf  «%-50s  %8s  %8s\n»  «------------------------------------------------»  «--------»
«--------»


total_exec=0
total_nonexec=0
missing=0


IFS=':' read -ra dirs <<< «$PATH»


for d in «${dirs[@]}»; do
    if [ -d «$d» ]; then
        e=$(find «$d» -maxdepth 1 -type f -executable 2>/dev/null | wc -l)
        ne=$(find «$d» -maxdepth 1 -type f ! -executable 2>/dev/null | wc -l)
        total_exec=$((total_exec + e))
```

```
            total_nonexec=$((total_nonexec + ne))
            printf «%-50s %8d %8d\n» «$d» «$e» «$ne»
        else
            missing=$((missing + 1))
            printf «%-50s %8s %8s\n» «$d» «-» «-»
        fi
    done


    echo «============================================================»
    printf «Каталогов в PATH:                  %d (из них не существует: %d)\n» «${#dirs[@]}»
«$missing»
    printf «Всего исполняемых файлов:      %d\n» «$total_exec»
    printf «Всего неисполняемых файлов:    %d\n» «$total_nonexec»
    printf «Всего файлов в PATH:           %d\n» «$((total_exec + total_nonexec))»
    echo «============================================================»
```

A.7 Сценарий вывода дерева каталогов с размерами файлов

```
    #!/bin/bash
    # tree_sizes.sh — вывод дерева каталогов с размерами (уровень 8)
    # Для каталогов — число файлов; для файлов — размер в KB/MB/GB и KiB/MiB/GiB
    # Использование: ./tree_sizes.sh [каталог]   (по умолчанию текущий)


    target=«${1:-.}»


    # Размер в десятичных единицах: KB = 1000 байт
    fmt_dec() {
        awk -v b=«$1» 'BEGIN {
            if (b < 1000)          printf «%d B», b
            else if (b < 1000000)   printf «%.2f KB», b/1000
            else if (b < 1000000000) printf «%.2f MB», b/1000000
            else                   printf «%.2f GB», b/1000000000
        }'
    }


    # Размер в двоичных единицах: KiB = 1024 байт
    fmt_bin() {
        awk -v b=«$1» 'BEGIN {
            if (b < 1024)            printf «%d B», b
            else if (b < 1048576)    printf «%.2f KiB», b/1024
            else if (b < 1073741824) printf «%.2f MiB», b/1048576
            else                     printf «%.2f GiB», b/1073741824
        }'
    }


    echo «================================================================»
    echo « ДЕРЕВО КАТАЛОГОВ: $target»
    echo « Формат размера файла: <десятичный> / <двоичный>»
    echo «================================================================»
    printf «%s[DIR ] %-40s файлов: %s, размер: %s / %s\n» «$2» «.» «0» «0 B» «0 B»


    print_tree() {
        local dir=«$1» indent=«$2»
        local fcount=0 tsize=0


        # Считаем рекурсивно по текущему каталогу
        while IFS= read -r -d '' f; do
            if [ -f «$f» ]; then
                fcount=$((fcount + 1))
                tsize=$((tsize + $(stat -c%s «$f»)))
            fi
        done < <(find «$dir» -type f -print0 2>/dev/null)


        local dsize dec_size bin_size
        dsize=$(fmt_dec «$tsize»)
        bin_size=$(fmt_bin «$tsize»)
          printf «%s[DIR ] %-40s файлов: %d, размер: %s / %s\n» «$indent» «$(basename »$dir«)»
«$fcount» «$dsize» «$bin_size»
```

```
    local item name
    for item in «$dir»/* «$dir»/.[!.]*; do
        [ -e «$item» ] || continue
        name=$(basename «$item»)
        if [ -d «$item» ]; then
            print_tree «$item» «$indent  »
        else
            local sz szd szb
            sz=$(stat -c%s «$item»)
            szd=$(fmt_dec «$sz»)
            szb=$(fmt_bin «$sz»)
            printf «%s[FILE] %-40s %s / %s\n» «$indent  » «$name» «$szd» «$szb»
        fi
    done
}


print_tree «$target» «»
echo «================================================================»
```

A.8 Сценарий монитора топ-5 процессов по загрузке процессора и памяти

```
#!/bin/bash
# top5.sh — монитор топ-5 процессов по CPU и памяти (уровень 9)
# Использование: ./top5.sh [интервал_в_секундах]   (по умолчанию 5)
# Работает в бесконечном цикле до Ctrl+C.


INTERVAL=«${1:-5}»


if ! [[ $INTERVAL =~ ^[0-9]+$ ]] || [ «$INTERVAL» -eq 0 ]; then
    echo «Ошибка: интервал должен быть положительным целым числом секунд»
    exit 1
fi


trap 'echo; echo «Монитор остановлен.»; exit 0' INT TERM


# Итоговая память системы в МБ — для перевода процентов в мегабайты
TOTAL_MB=$(awk '/MemTotal/ {printf «%d», $2/1024}' /proc/meminfo)


fmt_row() {
    printf «%-8s %-24s %8s %10s %8s\n» «$1» «$2» «$3» «$4» «$5»
}


while true; do
    clear
    echo «================================================================»
    echo « МОНИТОР ПРОЦЕССОВ - ТОП-5»
    echo « Время:        $(date '+%Y-%m-%d %H:%M:%S')»
    echo « Интервал:     ${INTERVAL} с (Ctrl+C - выход)»
    echo « Всего памяти: ${TOTAL_MB} МБ»
    echo «================================================================»


    echo
    echo « ТОП-5 ПРОЦЕССОВ ПО ЗАГРУЗКЕ ЦП»
    fmt_row «PID» «ИМЯ» «CPU,%» «ПАМЯТЬ,МБ» «ВРЕМЯ CPU»
    fmt_row «--------» «------------------------» «--------» «---------» «----------»
     ps -eo pid,comm,%cpu,%mem,time --sort=-%cpu 2>/dev/null | head -6 | tail -5 | while read
pid comm cpu mem cputime; do
        mbm=$(awk -v p=«$mem» -v t=«$TOTAL_MB» 'BEGIN{printf «%.1f», p*t/100}')
        printf «%-8s %-24s %8s %10s %8s\n» «$pid» «$comm» «$cpu» «$mbm» «$cputime»
    done


    echo
    echo « ТОП-5 ПРОЦЕССОВ ПО ЗАГРУЗКЕ ОПЕРАТИВНОЙ ПАМЯТИ»
    fmt_row «PID» «ИМЯ» «CPU,%» «ПАМЯТЬ,МБ» «ВРЕМЯ CPU»
    fmt_row «--------» «------------------------» «--------» «---------» «----------»
     ps -eo pid,comm,%cpu,%mem,time --sort=-%mem 2>/dev/null | head -6 | tail -5 | while read
pid comm cpu mem cputime; do
        mbm=$(awk -v p=«$mem» -v t=«$TOTAL_MB» 'BEGIN{printf «%.1f», p*t/100}')
        printf «%-8s %-24s %8s %10s %8s\n» «$pid» «$comm» «$cpu» «$mbm» «$cputime»
```

```
    done

    echo
    echo «================================================================»
    sleep «$INTERVAL»
done
```

A.9 Сценарий подготовки исходной структуры рабочего каталога

```
#!/bin/bash
# prep.sh — подготовка среды для ЛР2 (Linux-эквивалент Prep v2.bat)
# Запускать из каталога OS2:  bash prep.sh
#
# Скрипт идемпотентен: перед созданием структуры снимает режимы «только для
# чтения» и удаляет каталоги предыдущего прогона. Без этого повторный запуск
# падал бы на chmod 444 и копирование не создавало бы файлы.


echo «Подготовка структуры ЛР2...»


# чистка предыдущего прогона: вернуть права и убрать каталоги
chmod -R u+w . 2>/dev/null
rm -rf task_01a task_01b task_01b_dst_1 task_01b_dst_2 task_01c_src task_01c_dst \
       task_01d_1 task_01d_2 task_01e task_01f task_01g task_01h task_01i task_01j task_01k \
       tasks_tmp
rm -f students.txt console.txt bios.txt


# ------------------------------------------------
# tasks_tmp — источник файлов для копирования
# ------------------------------------------------
mkdir -p tasks_tmp
echo txt1 > tasks_tmp/txt1.txt
echo txt2 > tasks_tmp/txt2.txt
echo txt3 > tasks_tmp/txt3.txt
echo hidden1 > tasks_tmp/hidden1.txt
echo system1 > tasks_tmp/system1.txt
touch tasks_tmp/bmp1.bmp tasks_tmp/bmp2.bmp
touch tasks_tmp/exe1.exe tasks_tmp/exe2.exe
chmod 755 tasks_tmp/exe1.exe tasks_tmp/exe2.exe   # аналог исполняемых


# ------------------------------------------------
# task_01a — приёмник для cp
# ------------------------------------------------
mkdir -p task_01a


# ------------------------------------------------
# task_01b — структура для cp -r (аналог xcopy)
# ------------------------------------------------
mkdir -p task_01b/subdir/empty_subdir
echo txt1 > task_01b/subdir/txt1.txt
echo txt2 > task_01b/subdir/txt2.txt
mkdir -p task_01b/hidden_dir/empty_subdir task_01b/hidden_dir/subdir
echo txt1 > task_01b/hidden_dir/subdir/txt1.txt
echo txt2 > task_01b/hidden_dir/subdir/txt2.txt
echo txt1 > task_01b/hidden_dir/txt1.txt
echo system1 > task_01b/hidden_dir/system1.txt
echo readonly > task_01b/readonly.txt          # аналог атрибута +r
chmod 444 task_01b/readonly.txt
echo archive > task_01b/archive-readonly.txt   # аналог +a +r
chmod 444 task_01b/archive-readonly.txt
echo plain > task_01b/plain.txt                # обычный, без атрибутов
chmod 644 task_01b/plain.txt


# ------------------------------------------------
# task_01c — источник и приёмник для mv
# ------------------------------------------------
mkdir -p task_01c_src task_01c_dst
for n in 1 2 3 4 5; do echo txt1 > task_01c_src/txt$n.txt; done


# ------------------------------------------------
# task_01d — источник и приёмник для замены файлов
```

```
# ------------------------------------------------
mkdir -p task_01d_1 task_01d_2
for n in 1 2 3; do
  cp tasks_tmp/txt$n.txt task_01d_1/
  cp tasks_tmp/txt$n.txt task_01d_2/
done


# ------------------------------------------------
# task_01e — файлы для переименования
# ------------------------------------------------
mkdir -p task_01e
for n in 1 2 3 4 5; do cp tasks_tmp/txt1.txt task_01e/txt$n.txt; done


# ------------------------------------------------
# task_01f — файлы для сравнения (diff/fc)
# ------------------------------------------------
mkdir -p task_01f
printf 'Start\nHello World\nEnd\n' > task_01f/txt1.txt
printf 'Start\nHello, World!\nEnd\n' > task_01f/txt2.txt


# ------------------------------------------------
# task_01g — структура для удаления (del/rm)
# system*.txt — аналог атрибута «Системный»: в Linux такого атрибута нет,
# поэтому «системные» файлы помечены именем, их удаляет задание 15.
# readonly.txt — аналог «Только для чтения», его обязана пережить задание 16.
# ------------------------------------------------
mkdir -p task_01g/subdir/empty_subdir
echo txt1 > task_01g/subdir/txt1.txt
echo txt2 > task_01g/subdir/txt2.txt
mkdir -p task_01g/hidden_dir/empty_subdir task_01g/hidden_dir/subdir
echo txt1 > task_01g/hidden_dir/subdir/txt1.txt
echo txt2 > task_01g/hidden_dir/subdir/txt2.txt
echo txt1 > task_01g/hidden_dir/txt1.txt
echo system1 > task_01g/hidden_dir/system1.txt
echo readonly > task_01g/readonly.txt
chmod 444 task_01g/readonly.txt
echo plain > task_01g/plain.txt
echo txt1 > task_01g/txt1.txt
echo system1 > task_01g/system1.txt
echo system2 > task_01g/system2.txt


# ------------------------------------------------
# task_01h — приёмник для вывода списка
# ------------------------------------------------
mkdir -p task_01h


# ------------------------------------------------
# task_01i — каталоги для навигации
# ------------------------------------------------
mkdir -p «task_01i/Русский» task_01i/English


# ------------------------------------------------
# task_01k — каталоги для удаления
# ------------------------------------------------
mkdir -p task_01k/subdir task_01k/empty_subdir
echo file > task_01k/subdir/file.txt


echo «================================================»
echo «Подготовка завершена.»
echo «================================================»
```